



INTEGRATED INTERNATIONAL STUDENT PORTAL
TEAM: SMO-A

Analysis and design report

Ahmed Abdinur Ahmed,
Paschal Chukwubuikem Ifewulu,
Favour Chinyere Okoye,
Alex Van Poppel,
Fanni Veh

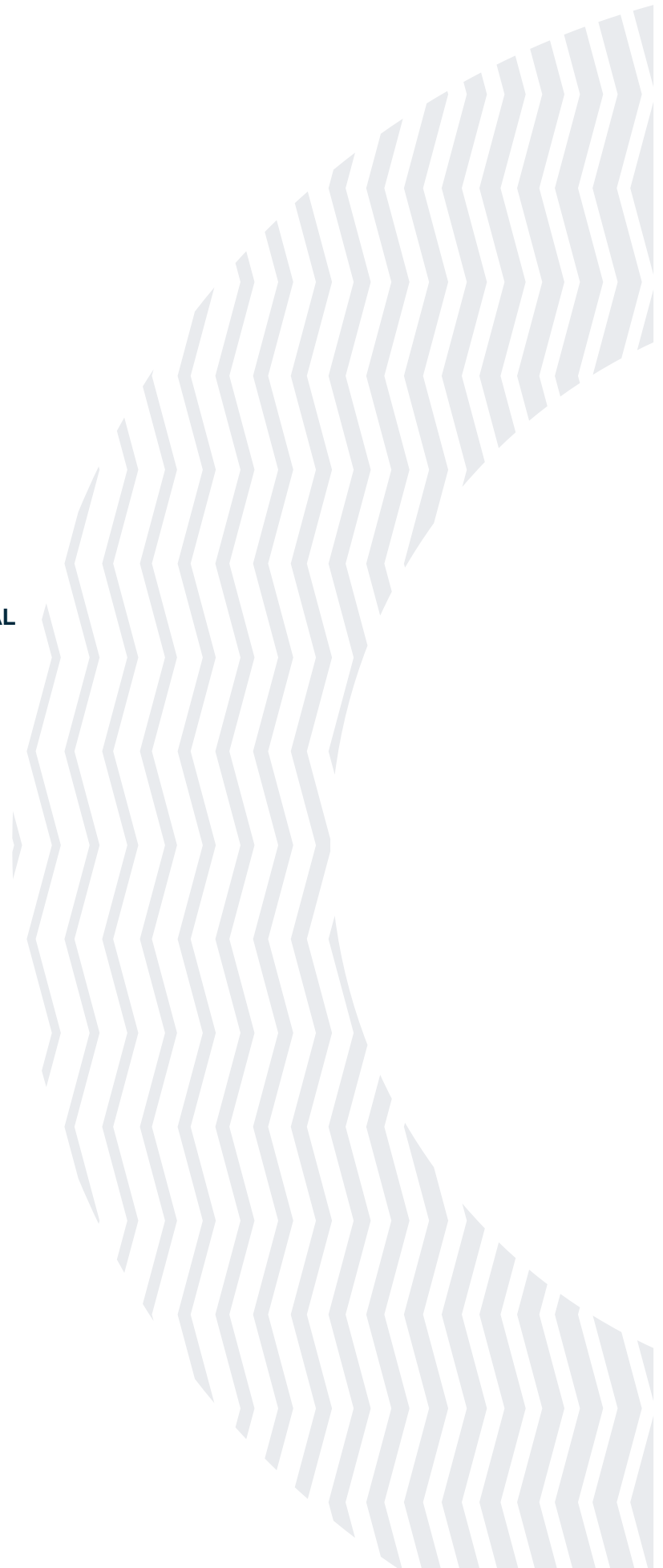


Table of contents

1. Introduction	4
2. Requirements analysis.....	5
2.1. Functional requirements.....	6
1.1.1. Use Case Diagram.....	6
1.1.2. Use Cases.....	7
1.1.2.1. Login.	7
1.1.2.2. View homepage.	7
1.1.2.3. Apply as buddy.	8
1.1.2.4. Apply as Landlord	9
1.1.2.5. Apply as home Institution.....	10
1.1.2.6. Fill in an application form.	11
1.1.2.7. Nominate student.....	12
1.1.2.8. Sign OLA.....	13
1.1.2.9. Approve house owners.	14
1.1.2.10. Approve students.	15
1.1.2.11. Manage guide.	16
1.1.2.12. Manage home owner.	17
1.1.2.13. Plan webinar.	18
1.1.2.14. Manage students.	19
1.1.2.15. Approve student profile.....	20
1.1.2.16. Sign OLA.....	21
1.1.2.17. Approve course enrollment.....	22
1.1.2.18. Reply to question.	23
1.1.2.19. Apply for a buddy.	23
1.1.2.20. Approve buddy.....	25
1.1.2.21. View guide	25
1.1.2.22. Apply for a bike	26
1.1.2.23. Apply for room.....	27
1.1.2.24. Ask a question.	28
1.1.2.25. Reply to question.	29
1.1.2.26. Send a message in board.....	30
1.1.2.27. Enroll in courses	31
1.1.2.28. Sign OLA.....	32
1.1.2.29. View students.....	33
1.1.2.30. Approve student.....	34
1.1.2.31. Manage room.....	35
1.1.2.32. Manage properties.....	36
1.1.2.33. Manage users.	37

1.1.2.34. Manage programs.....	38
2. Data Model.....	39
3. User Stories	42

1. Introduction

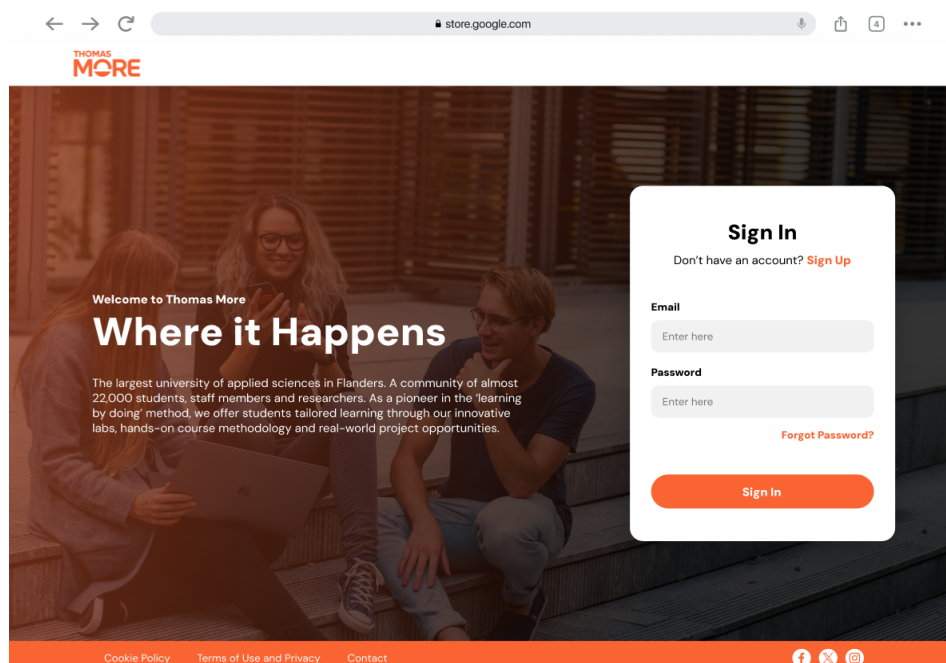
Navigating the complexities of studying abroad can be both an exhilarating and daunting experience for international students. The process often involves a myriad of administrative tasks that can lead to confusion, delays, and added stress for students and coordinators alike. Our initiative aims to address these challenges by developing a comprehensive digital platform designed to streamline the entire application and enrollment process for international students, whether they are pursuing a full degree or participating in an exchange program.

The proposed system, the Integrated International Student Portal, seeks to simplify the administrative workflow by consolidating essential functions into a single, user-friendly interface. This platform will enhance collaboration among key stakeholders, including students, home institutions, international offices, and housing providers. By offering clear guidance and resources, the portal will empower students to navigate their journey with confidence and ease.

Key features of the portal include:

- **Centralized Application Process:** A streamlined approach to submitting applications, managing documentation, and tracking progress.
- **Buddy Program Integration:** A feature that connects new students with experienced peers for support and guidance.
- **Housing Assistance:** Tools to facilitate the search for accommodation and manage housing options.
- **Communication Tools:** Integrated messaging systems to ensure effective communication among students, fostering connections and collaboration.

This outlines the functional requirements of the Integrated International Student Portal, highlighting its potential to reduce administrative burdens, improve user experience, and facilitate a smoother transition for international students at our institution. By prioritizing user needs, we aim to create an environment where students can concentrate on their academic and cultural experiences without being burdened by administrative complexities.



2. Requirements analysis

This section presents a comprehensive analysis of the functional and non-functional requirements for the Integrated International Student Portal. The goal is to provide clarity on the capabilities and features necessary to address the needs of international and exchange students, as well as the stakeholders involved in their academic journey.

The portal is designed to facilitate a seamless application and enrollment process while reducing the administrative burden on coordinators. To achieve this, we will begin with a use case diagram that identifies the various user roles—both active and passive—and outlines the actions each can perform within the system. This visual representation will serve as a foundation for further discussion on each requirement.

We will detail the functions required for:

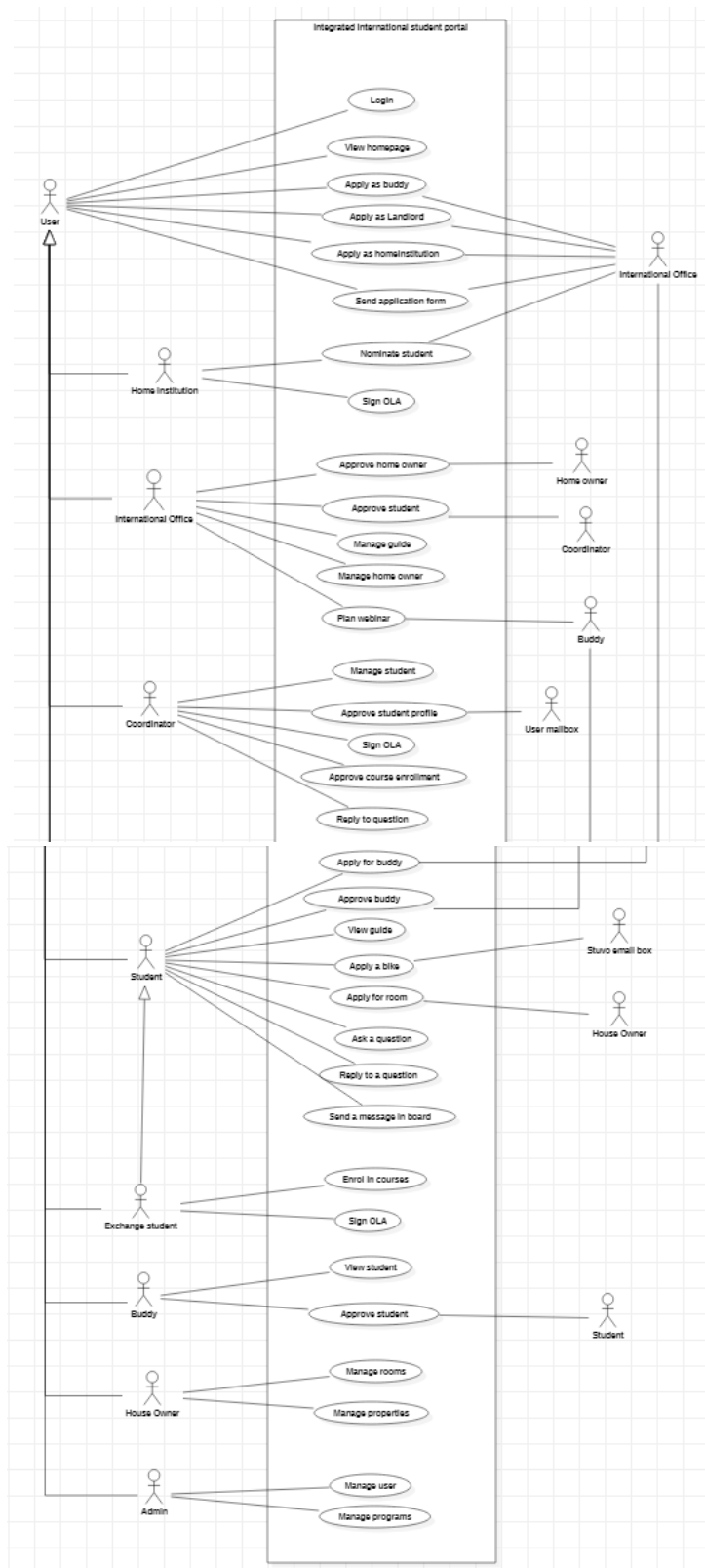
- **Application Processing:** This includes the nomination of students, registration at Thomas More, application submissions, and the creation of the Online Learning Agreement (OLA).
- **Communication Features:** The platform must allow students to easily communicate and interact with each other, fostering connections and support among peers.
- **Housing and Transportation Options:** The portal must showcase available housing options and provide students with the ability to rent bikes through Stuvo.
- **Support and Follow-up Mechanisms:** There should be provisions for international coordinators to monitor the progress of students to ensure they receive any necessary assistance.

In addition to these functional requirements, this section will also address essential non-functional requirements, such as security, usability, and scalability. The system must be intuitive to navigate, secure to protect personal data, and capable of adapting to a growing user base.

By outlining these requirements, we aim to ensure that the Integrated International Student Portal not only meets the immediate needs of users but also lays the groundwork for future enhancements and growth. This approach will create a robust, user-centric experience that supports international students throughout their academic endeavors.

2.1. Functional requirements

1.1.1. Use Case Diagram



1.1.2. Use Cases

1.1.2.1. Login.

Functionality: As a user, I can login

Precondition: Has the login info for an account. Has been verified.

Data model view:

User
userId: int PK userFirstName: string NNA userLastName: string NNA email: string NNA password: string NNA institutionNameId: int FK phoneNumber: string NNA nationalityId: int FK4 NNA DTR genderId: int FK1 NA DTR startSemester: date NA exchangeStudent: bool NA buddy: bool NA socialSecurityNumber: string AK1 NA countryId: int FK5 NA DTR homeInstitutionId: int FK35 NA DTR programId: int FK16 NA DTN coordinatorId: int FK8 NA DTN funFact: string NA hobbies: string NA rNumber: string NA roleId: int NNA FK6 DTR profileStatus: bool NNA erasmusCode: string NA institutionUserID: int NA FK7 DTR

Link to prototype: [SMO-A New - User](#)

1.1.2.2. View homepage.

Functionality: As a user, I can view homepage

Link to prototype: [SMO-A New - User](#)

1.1.2.3. Apply as buddy.

Functionality: As a user, I can apply as a buddy

Data model view:

User
userId: int PK userFirstName: string NNA userLastName: string NNA email: string NNA password: string NNA institutionNameId: int FK phoneNumber: string NNA nationalityId: int FK4 NNA DTR genderId: int FK1 NA DTR startSemester: date NA exchangeStudent: bool NA buddy: bool NA socialSecurityNumber: string AK1 NA countryId: int FK5 NA DTR homeInstitutionId: int FK35 NA DTR programId: int FK16 NA DTN coordinatorId: int FK8 NA DTN funFact: string NA hobbies: string NA rNumber: string NA roleId: int NNA FK6 DTR profileStatus: bool NNA erasmusCode: string NA institutionUserID: int NA FK7 DTR

Link to prototype: [SMO-A New - User](#)

Data model actions:

- **Initial state:** When the page is loaded nothing is immediately queried. A form with information that is hardcoded is displayed to the user to fill in.
 - **Submit button is clicked:** After the submit button is clicked we **CREATE** a new record in the "User" table based on the information submitted by the student, with the attribute "profileStatus" set to false to signify the profile has not yet been approved and with the roleId set to that of the Buddy.

1.1.2.4. Apply as Landlord

Functionality: As a house owner, I can apply as a landlord

Data model view:

User
userId: int PK userFirstName: string NNA userLastName: string NNA email: string NNA password: string NNA institutionNameId: int FK phoneNumber: string NNA nationalityId: int FK4 NNA DTR genderId: int FK1 NA DTR startSemester: date NA exchangeStudent: bool NA buddy: bool NA socialSecurityNumber: string AK1 NA countryId: int FK5 NA DTR homeInstitutionId: int FK35 NA DTR programId: int FK16 NA DTN coordinatorId: int FK8 NA DTN funFact: string NA hobbies: string NA rNumber: string NA roleId: int NNA FK6 DTR profileStatus: bool NNA erasmusCode: string NA institutionUserID: int NA FK7 DTR

Link to prototype: [SMO-A New - User](#)

1.1.2.5. Apply as home Institution

Functionality: As a home institution, I can apply as a home institution

Data model view:

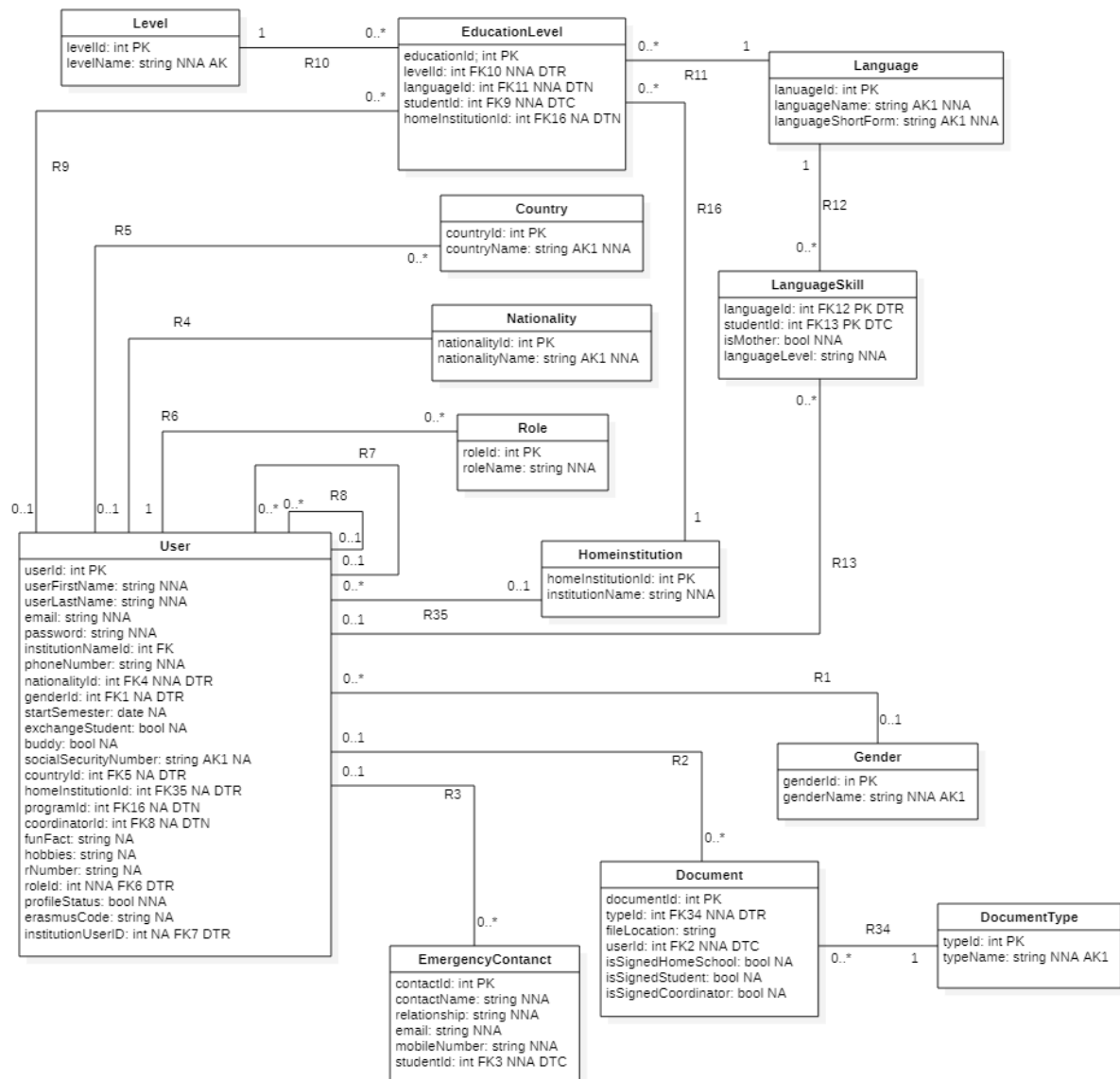
User
userId: int PK userFirstName: string NNA userLastName: string NNA email: string NNA password: string NNA institutionNameId: int FK phoneNumber: string NNA nationalityId: int FK4 NNA DTR genderId: int FK1 NA DTR startSemester: date NA exchangeStudent: bool NA buddy: bool NA socialSecurityNumber: string AK1 NA countryId: int FK5 NA DTR homeInstitutionId: int FK35 NA DTR programId: int FK16 NA DTN coordinatorId: int FK8 NA DTN funFact: string NA hobbies: string NA rNumber: string NA roleId: int NNA FK6 DTR profileStatus: bool NNA erasmusCode: string NA institutionUserID: int NA FK7 DTR

Link to prototype: [SMO-A New - User](#)

1.1.2.6. Fill in an application form.

Functionality: As a user, I am able to send in an application form to apply.

Data model view:



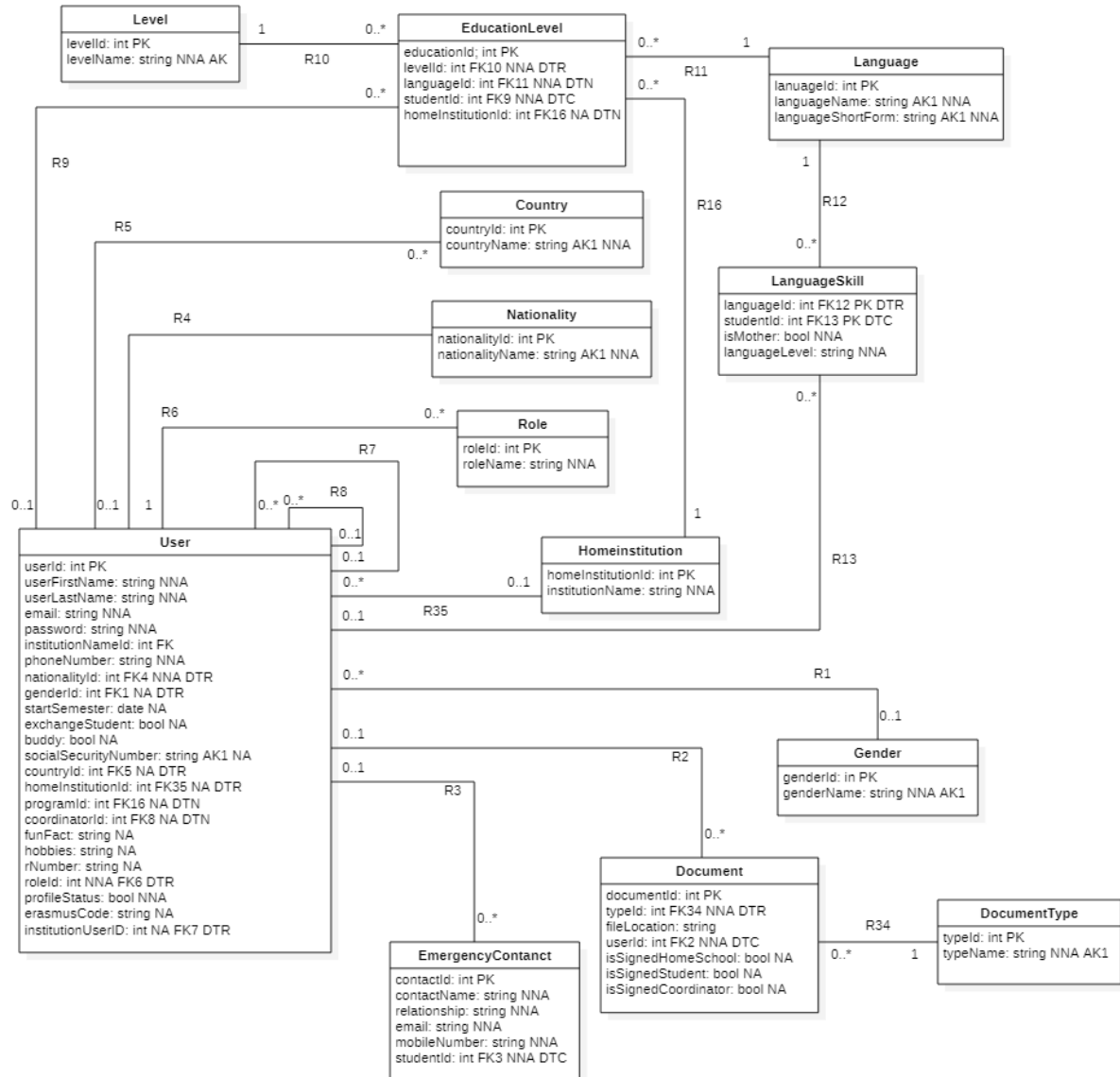
Link to prototype: [SMO-A New - User](#)

1.1.2.7. Nominate student.

Functionality: As a home institution, I can nominate students.

Precondition: Logged in as home institution.

Data model view:



Link to prototype: [SMO-A New - Home Institution](#)

Data model actions:

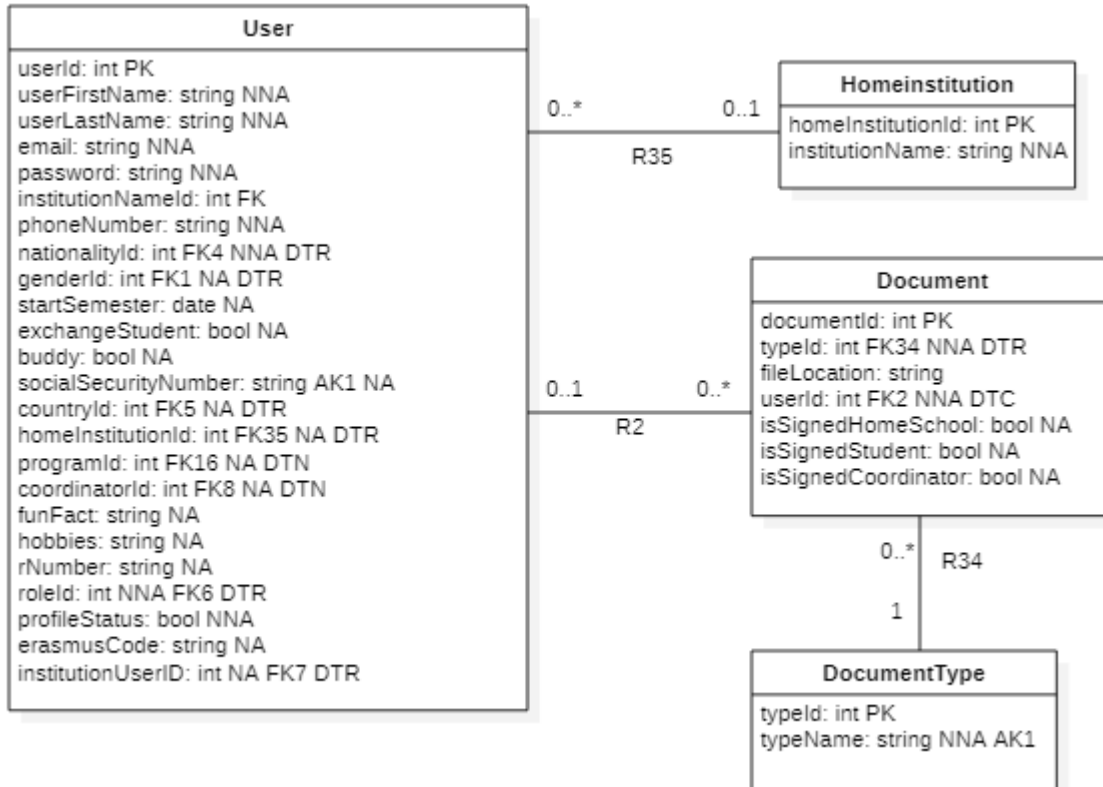
- **Initial state:** Hardcoded form for student (and home institution) information
 - **Submitting filled in document:** **UPDATE** all information shared about the user in classes User, Country, Nationality, EducationLevel, LanguageSkill and HomeInstitution.

1.1.2.8. Sign OLA.

Functionality: As a home institution, I can sign an OLA.

Precondition: Logged in as a home institution, OLA available to sign.

Data model view:



Link to prototype: [SMO-A New - Home Institution](#)

1.1.2.9. Approve house owners.

Functionality: As an international office, I can approve house owners.

Precondition: Logged in as an international office.

Data model view:

User
userId: int PK userFirstName: string NNA userLastName: string NNA email: string NNA password: string NNA institutionNameId: int FK phoneNumber: string NNA nationalityId: int FK4 NNA DTR genderId: int FK1 NA DTR startSemester: date NA exchangeStudent: bool NA buddy: bool NA socialSecurityNumber: string AK1 NA countryId: int FK5 NA DTR homeInstitutionId: int FK35 NA DTR programId: int FK16 NA DTN coordinatorId: int FK8 NA DTN funFact: string NA hobbies: string NA rNumber: string NA roleId: int NNA FK6 DTR profileStatus: bool NNA erasmusCode: string NA institutionUserID: int NA FK7 DTR

Link to prototype: [SMO-A New - International office](#)

1.1.2.10. *Approve students.*

Functionality: As an international office, I can approve of students.

Precondition: Logged in as an international office.

Data model view:

User
userId: int PK userFirstName: string NNA userLastName: string NNA email: string NNA password: string NNA institutionNameId: int FK phoneNumber: string NNA nationalityId: int FK4 NNA DTR genderId: int FK1 NA DTR startSemester: date NA exchangeStudent: bool NA buddy: bool NA socialSecurityNumber: string AK1 NA countryId: int FK5 NA DTR homeInstitutionId: int FK35 NA DTR programId: int FK16 NA DTN coordinatorId: int FK8 NA DTN funFact: string NA hobbies: string NA rNumber: string NA roleId: int NNA FK6 DTR profileStatus: bool NNA erasmusCode: string NA institutionUserID: int NA FK7 DTR

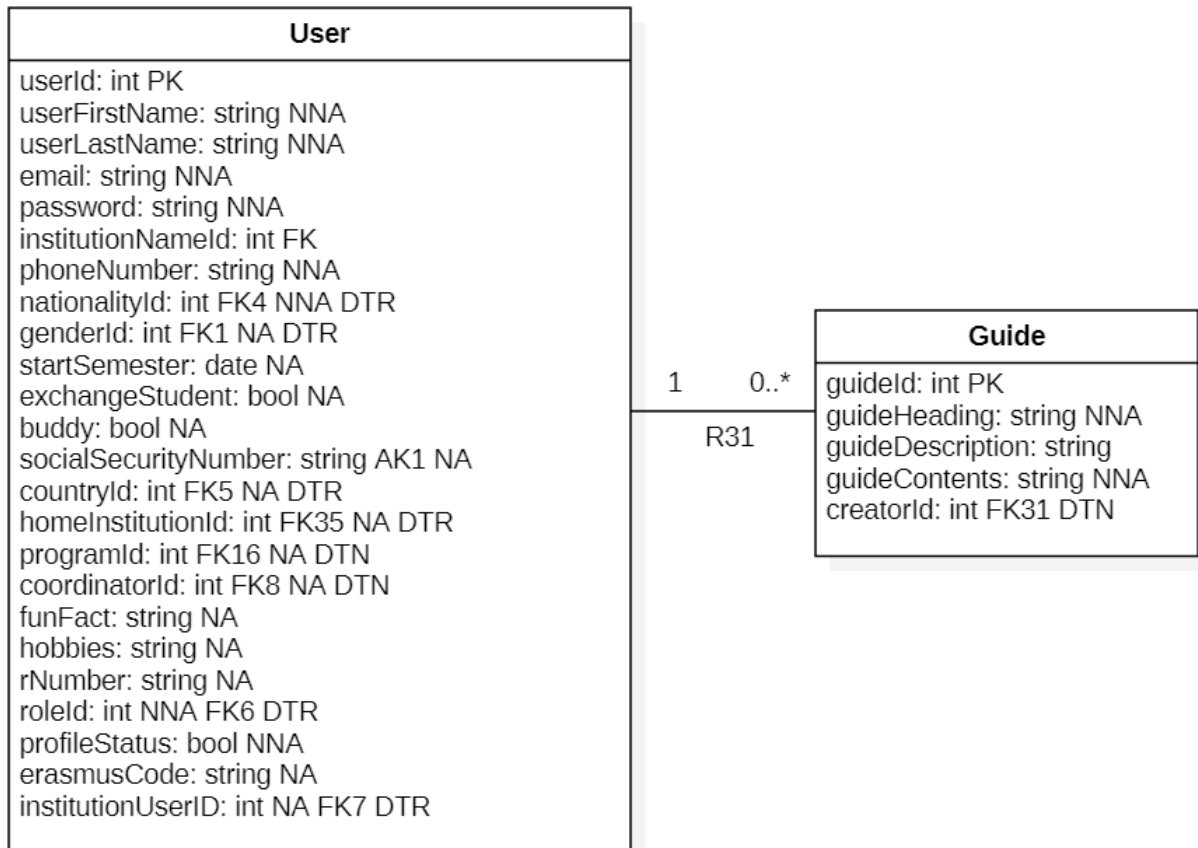
Link to prototype: [SMO-A New - International office](#)

1.1.2.11. *Manage guide.*

Functionality: As an international office, I can manage the guide.

Precondition: Logged in as an international office.

Data model view:



Link to prototype: [SMO-A New - International office](#)

1.1.2.12. *Manage home owner.*

Functionality: As an international office, I am able to manage home owners.

Precondition: Logged in as international office employee.

Data model view:

User
userId: int PK userFirstName: string NNA userLastName: string NNA email: string NNA password: string NNA institutionNameId: int FK phoneNumber: string NNA nationalityId: int FK4 NNA DTR genderId: int FK1 NA DTR startSemester: date NA exchangeStudent: bool NA buddy: bool NA socialSecurityNumber: string AK1 NA countryId: int FK5 NA DTR homeInstitutionId: int FK35 NA DTR programId: int FK16 NA DTN coordinatorId: int FK8 NA DTN funFact: string NA hobbies: string NA rNumber: string NA roleId: int NNA FK6 DTR profileStatus: bool NNA erasmusCode: string NA institutionUserID: int NA FK7 DTR

Link to prototype: [SMO-A New - International office](#)

1.1.2.13. *Plan webinar.*

Functionality: As an international office, I can plan a webinar.

Precondition: Logged in as international office employee.

Data model view:

User
userId: int PK userFirstName: string NNA userLastName: string NNA email: string NNA password: string NNA institutionNameId: int FK phoneNumber: string NNA nationalityId: int FK4 NNA DTR genderId: int FK1 NA DTR startSemester: date NA exchangeStudent: bool NA buddy: bool NA socialSecurityNumber: string AK1 NA countryId: int FK5 NA DTR homeInstitutionId: int FK35 NA DTR programId: int FK16 NA DTN coordinatorId: int FK8 NA DTN funFact: string NA hobbies: string NA rNumber: string NA roleId: int NNA FK6 DTR profileStatus: bool NNA erasmusCode: string NA institutionUserID: int NA FK7 DTR

Link to prototype: [SMO-A New - International office](#)

Data model actions:

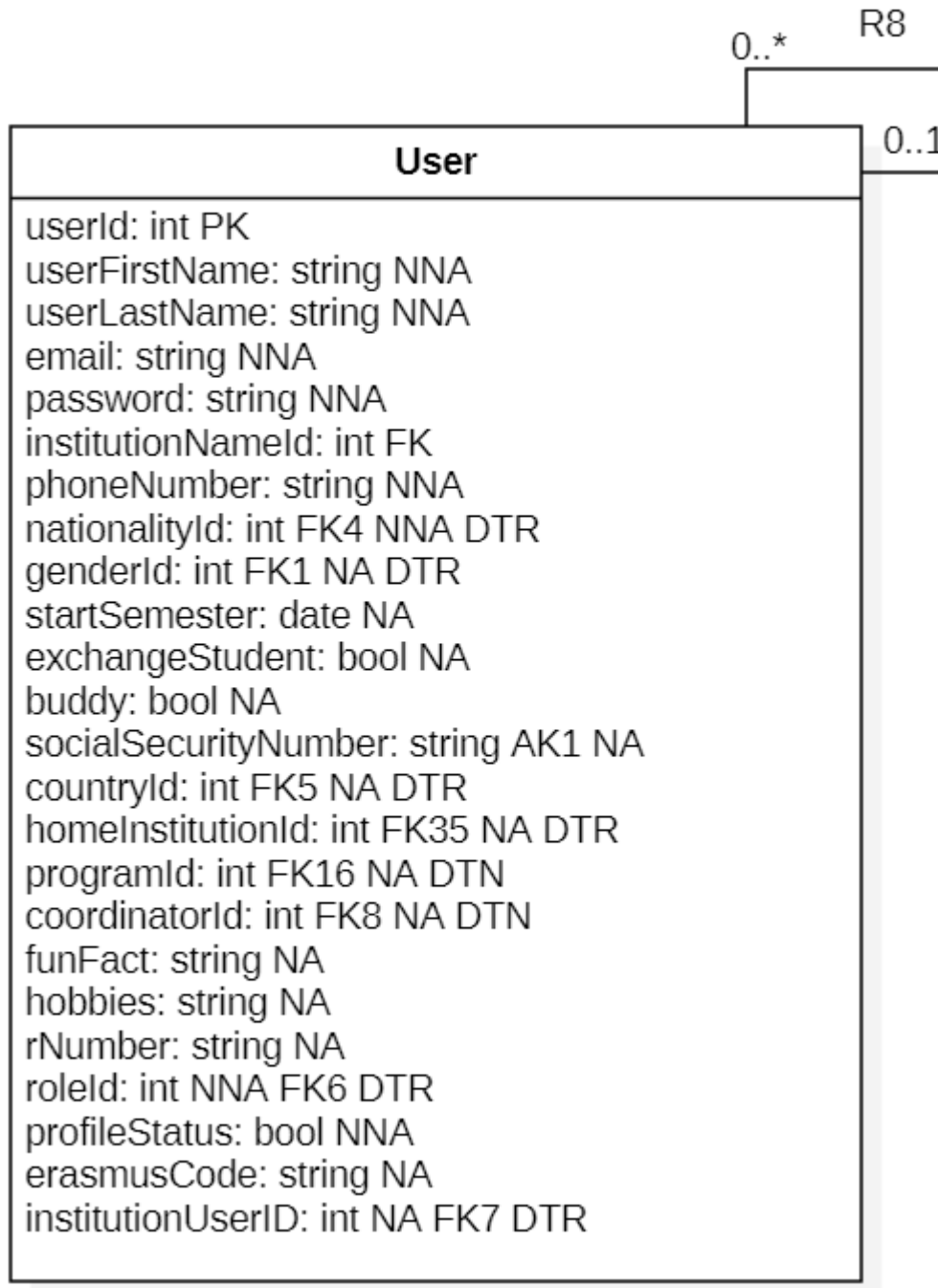
- **Initial state:** **READ** email addresses of all incoming students and hardcoded form for creating a webinar email
 - **Clicking send:** Emails sent to all users that had their emails ticked

1.1.2.14. *Manage students.*

Functionality: As a coordinator, I can manage students.

Precondition: Logged in as a coordinator

Data model view:



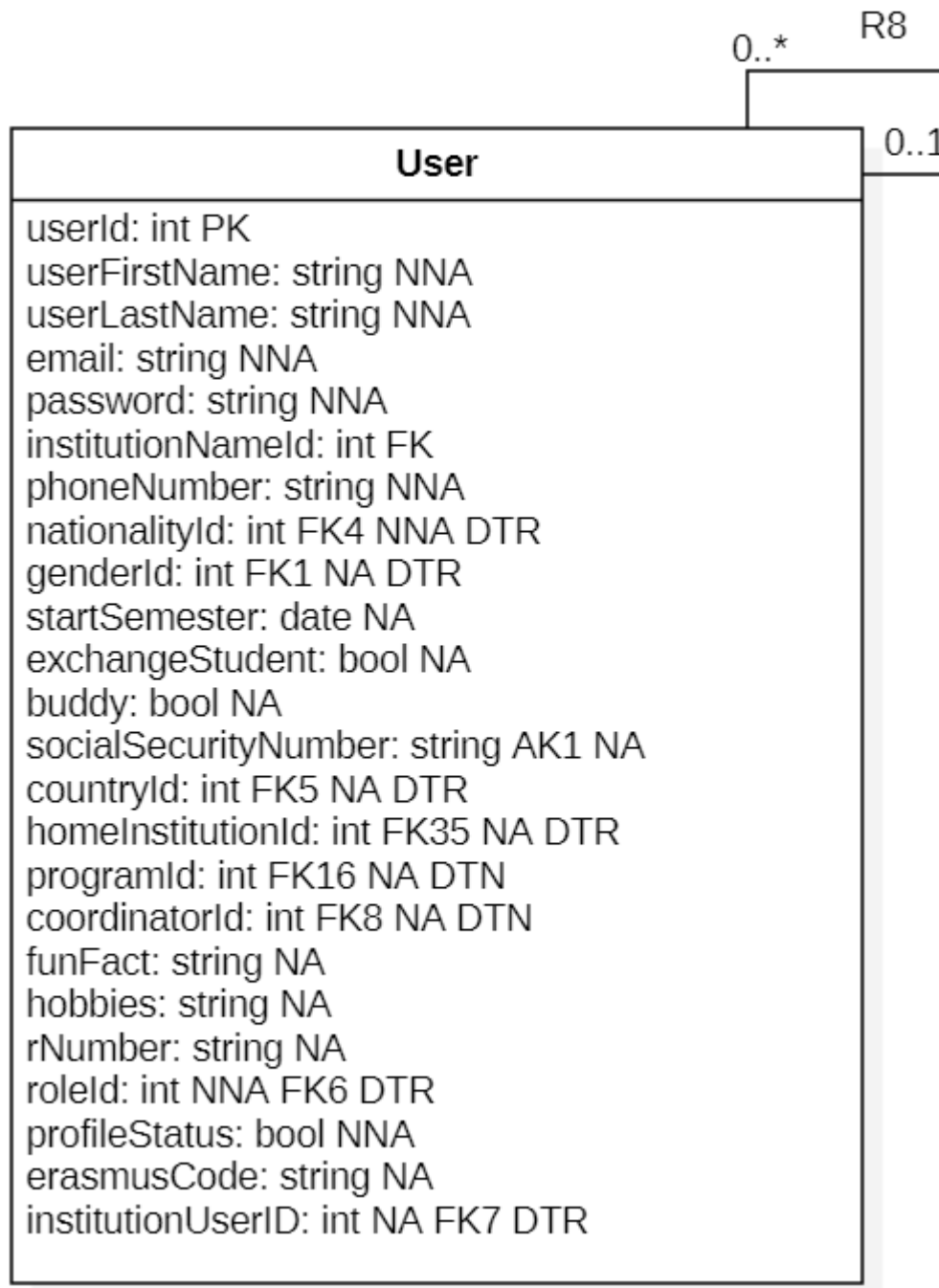
Link to prototype: [SMO-A New - Coordinator](#)

1.1.2.15. *Approve student profile.*

Functionality: As a coordinator, I can approve student profiles.

Precondition: Logged in as coordinator.

Data model view:



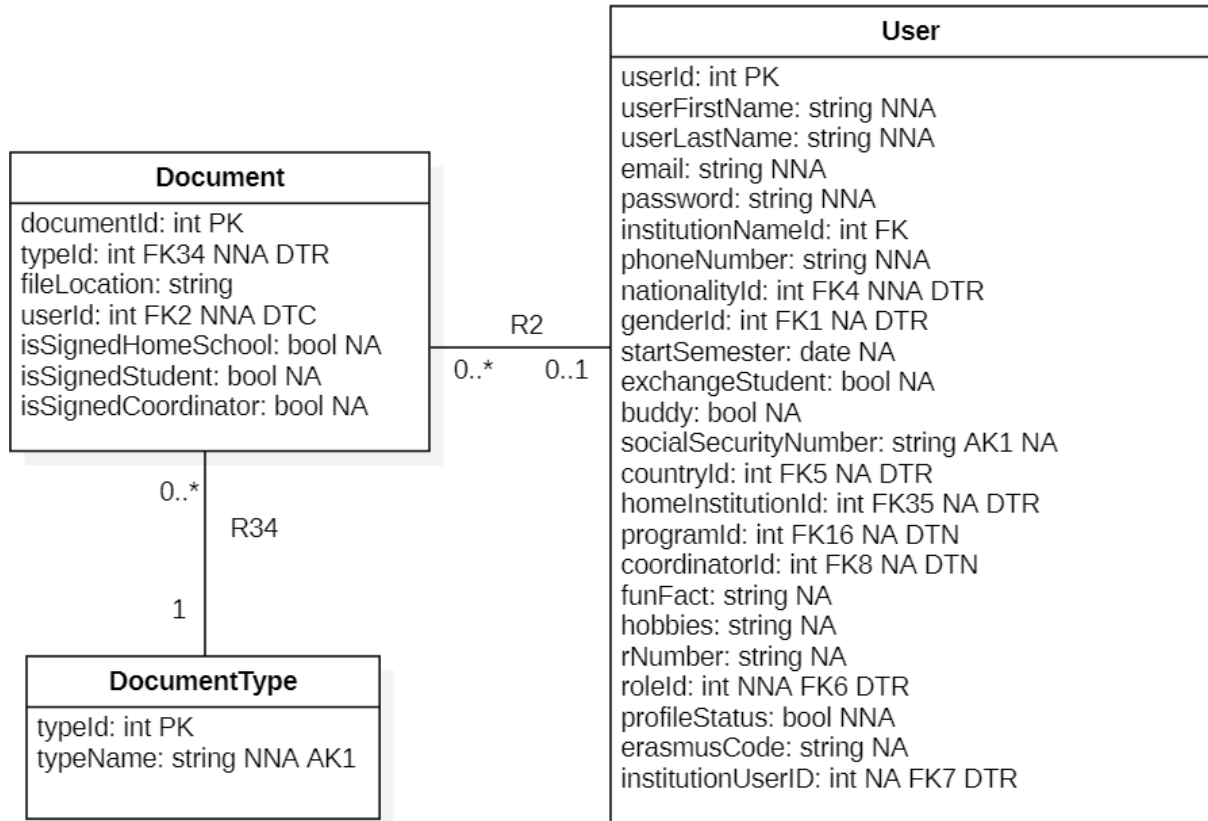
Link to prototype: [SMO-A New - Coordinator](#)

1.1.2.16. Sign OLA.

Functionality: As a coordinator, I can sign OLA.

Precondition: Logged in as coordinator, OLA available to sign.

Data model view:



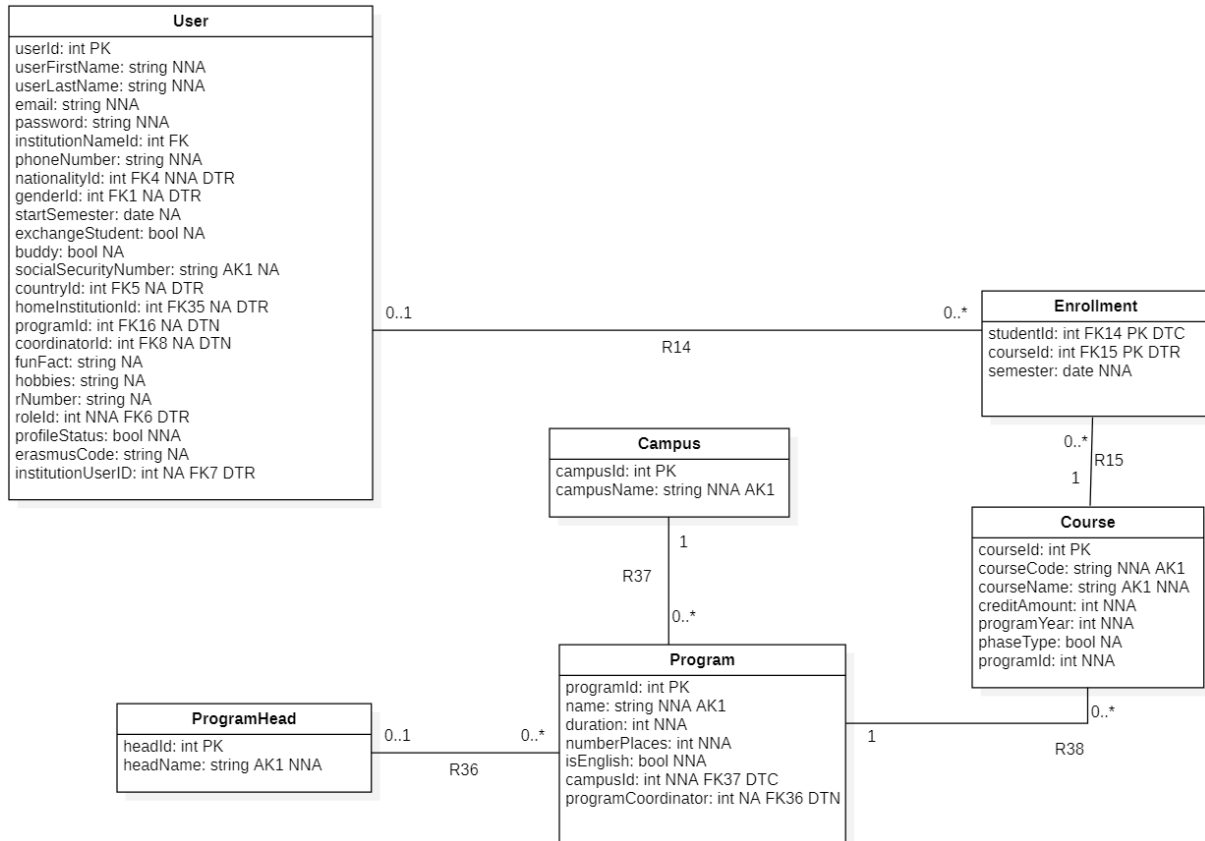
Link to prototype: [SMO-A New - Coordinator](#)

1.1.2.17. Approve course enrollment.

Functionality: As a coordinator, I can approve a student's course enrollment.

Precondition: Logged in as coordinator, student has sent in their course enrollment

Data model view:



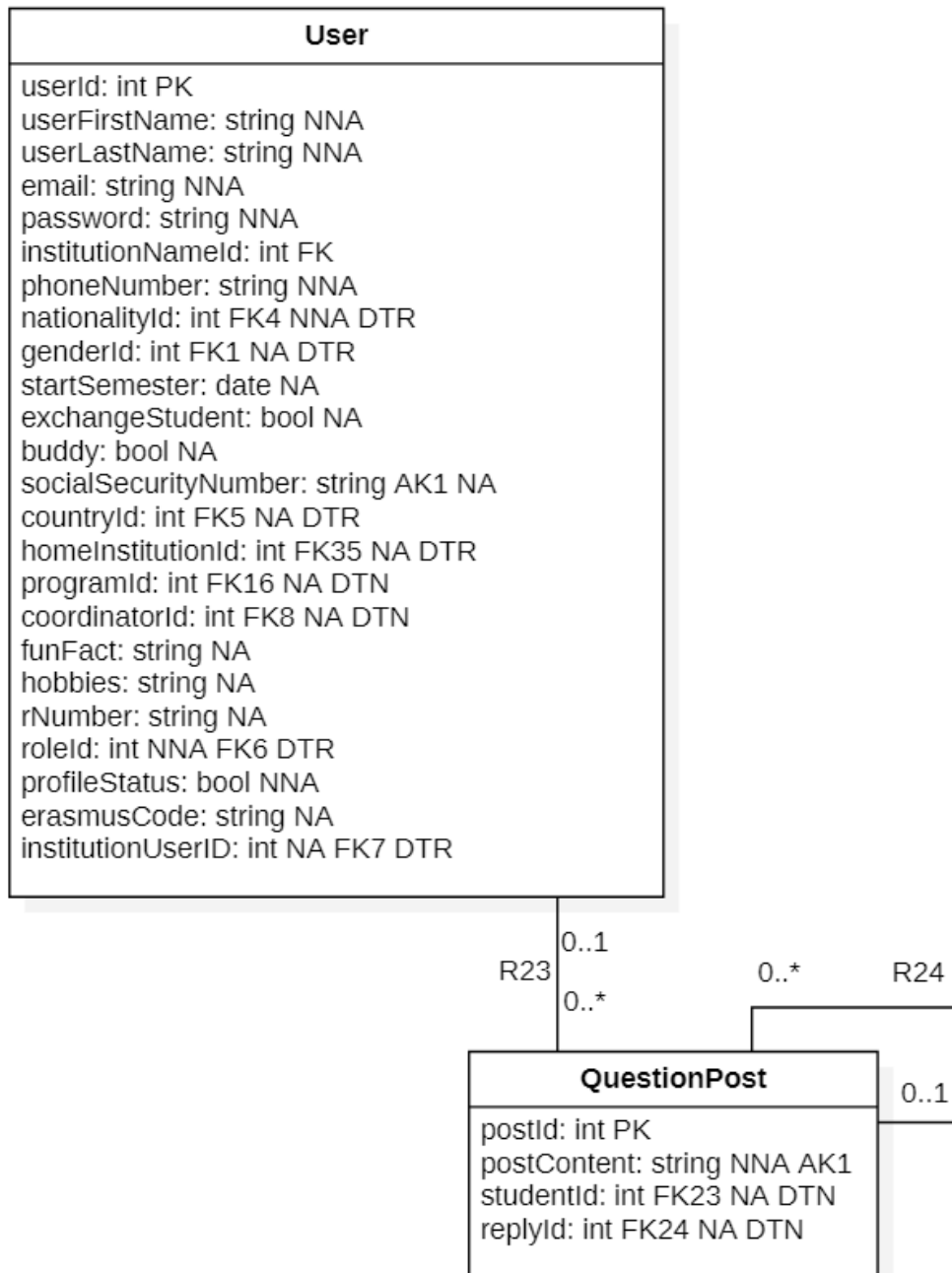
Link to prototype: [SMO-A New - Coordinator](#)

1.1.2.18. *Reply to question.*

Functionality: As a coordinator, I can reply to the message of a student

Precondition: Is logged in as a coordinator, Student has asked a question.

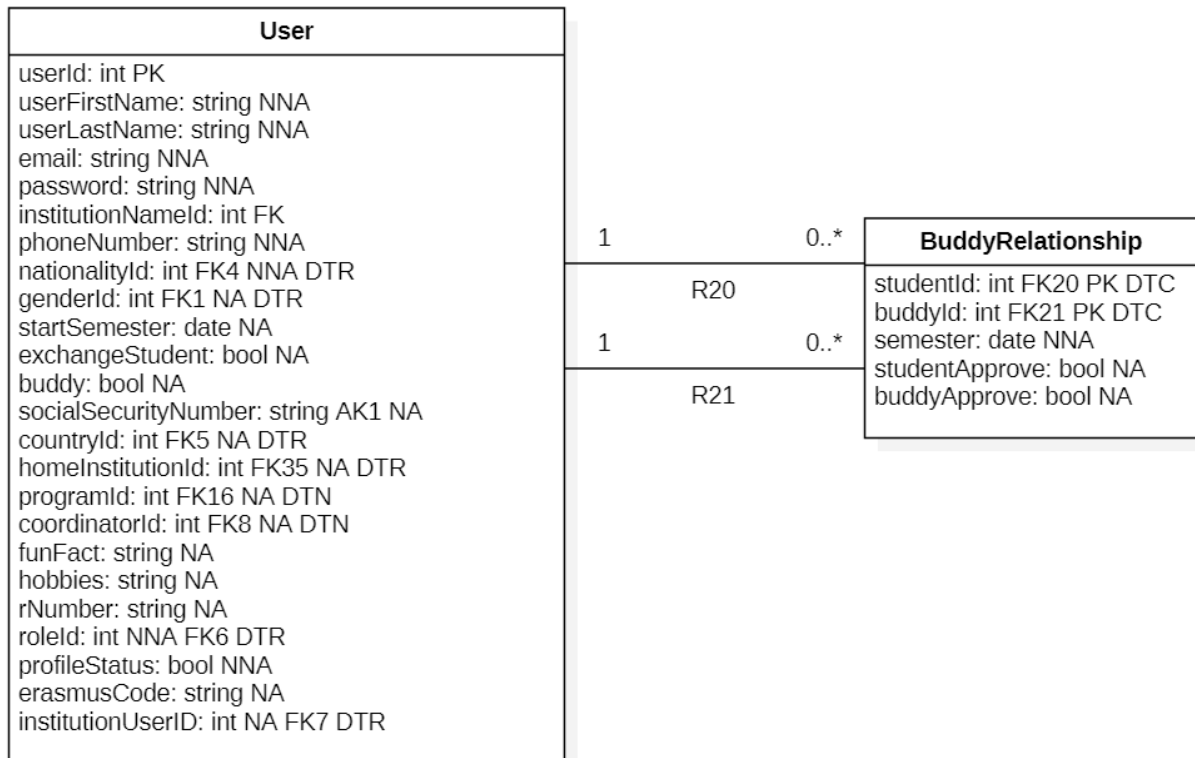
Data model view:



Link to prototype: [SMO-A New - Coordinator](#)

1.1.2.19. *Apply for a buddy.*

Functionality: As a student, I can apply for a buddy.

Data model view:

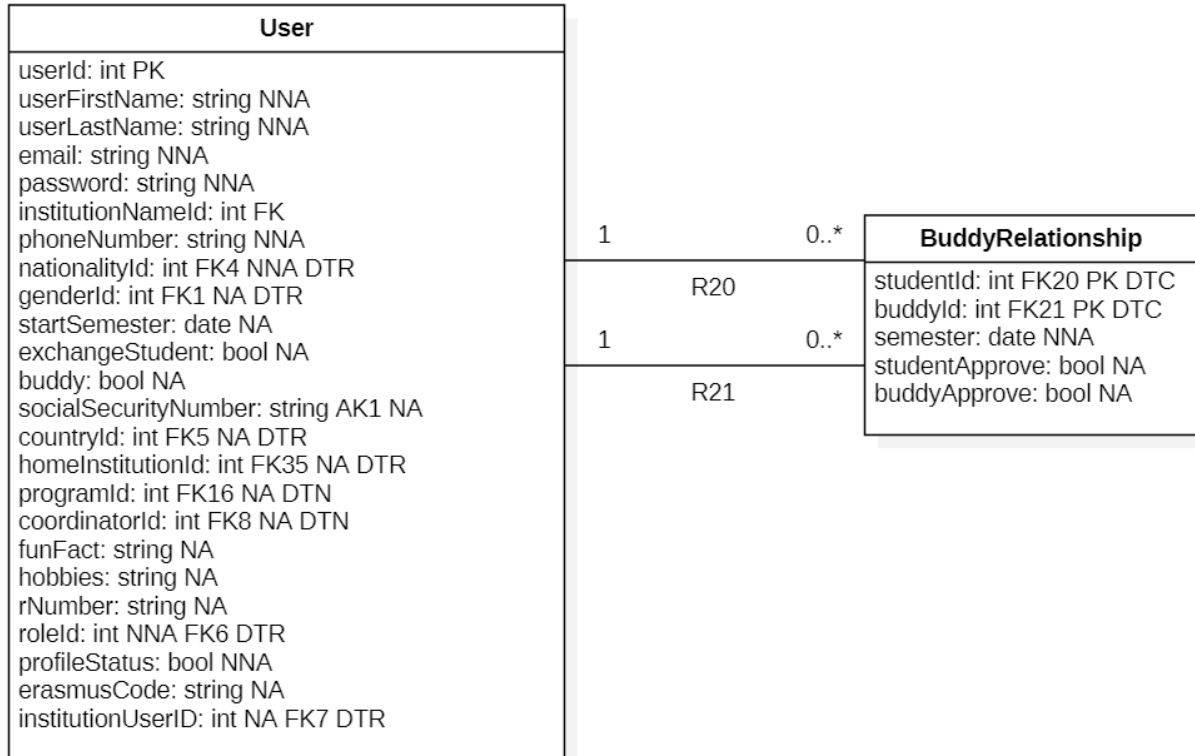
Link to prototype: [SMO-A New - Student](#)

1.1.2.20. Approve buddy

Functionality: As a student, I can approve a request to be my buddy.

Precondition: Logged in as student, and has requested a buddy.

Data model view:



Link to prototype: [SMO-A New - Student](#)

1.1.2.21. *View guide*

Functionality: As a student, I can view guides.

Precondition: Logged in as student.

Data model view:

Guide
guideId: int PK guideHeading: string NNA guideDescription: string guideContents: string NNA creatorId: int FK31 DTN

Link to prototype: [SMO-A New - Student](#)

1.1.2.22. *Apply for a bike*

Functionality: As a student, I can apply for a bike.

Precondition: Logged as student.

Data model view:

STUVO
stuvold: int PK stuvoEmail: string NNA AK

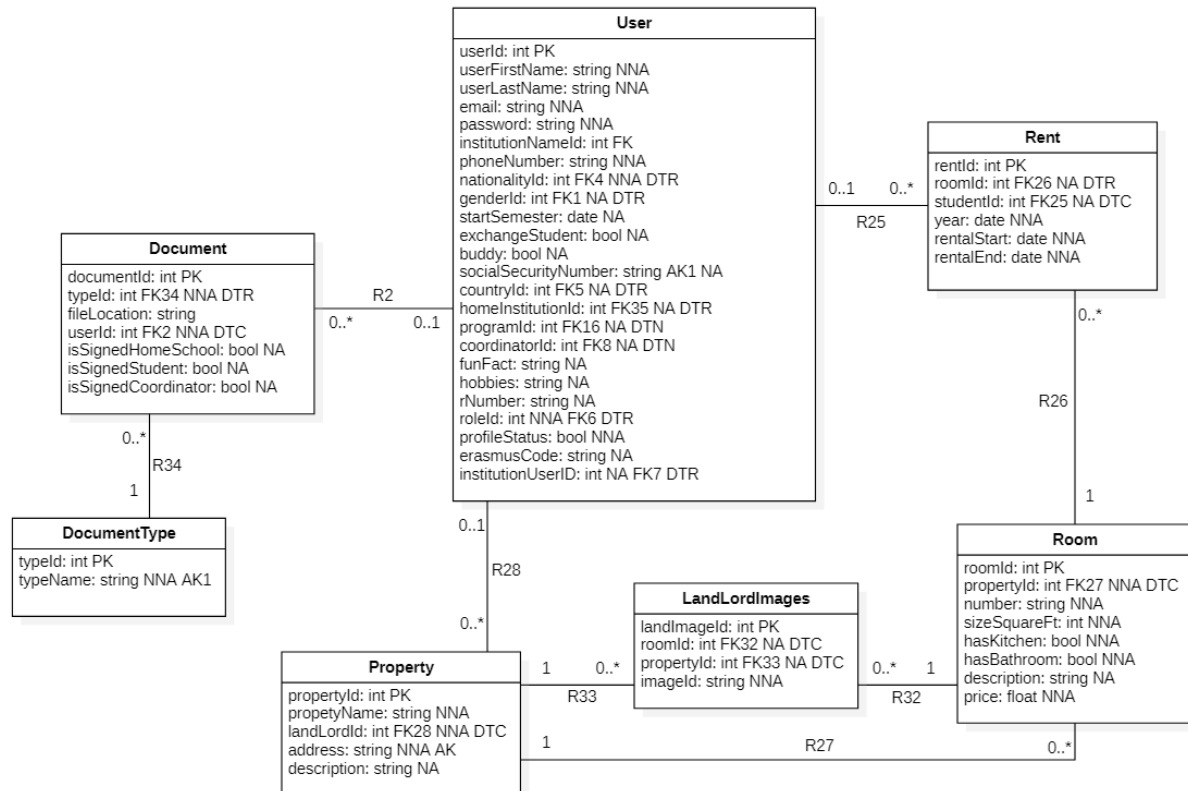
Link to prototype: [SMO-A New - Student](#)

1.1.2.23. Apply for room

Functionality: As a student, I can apply for a room.

Precondition: Logged in as student.

Data model view:



Link to prototype: [SMO-A New - Student](#)

Data model actions:

- **Initial state:** We **READ** the “User”, “Document”, and “DocumentType” tables for specific account details like name and profile picture.

For the main page content we **READ** the “rent” table for which rooms are available based on the “studentId” and “approvedRent” attributes. With a list of those available rooms we **READ** the “Room”, “Property”, and “LandLordImage” table for information specific to every property like address, images, etc.

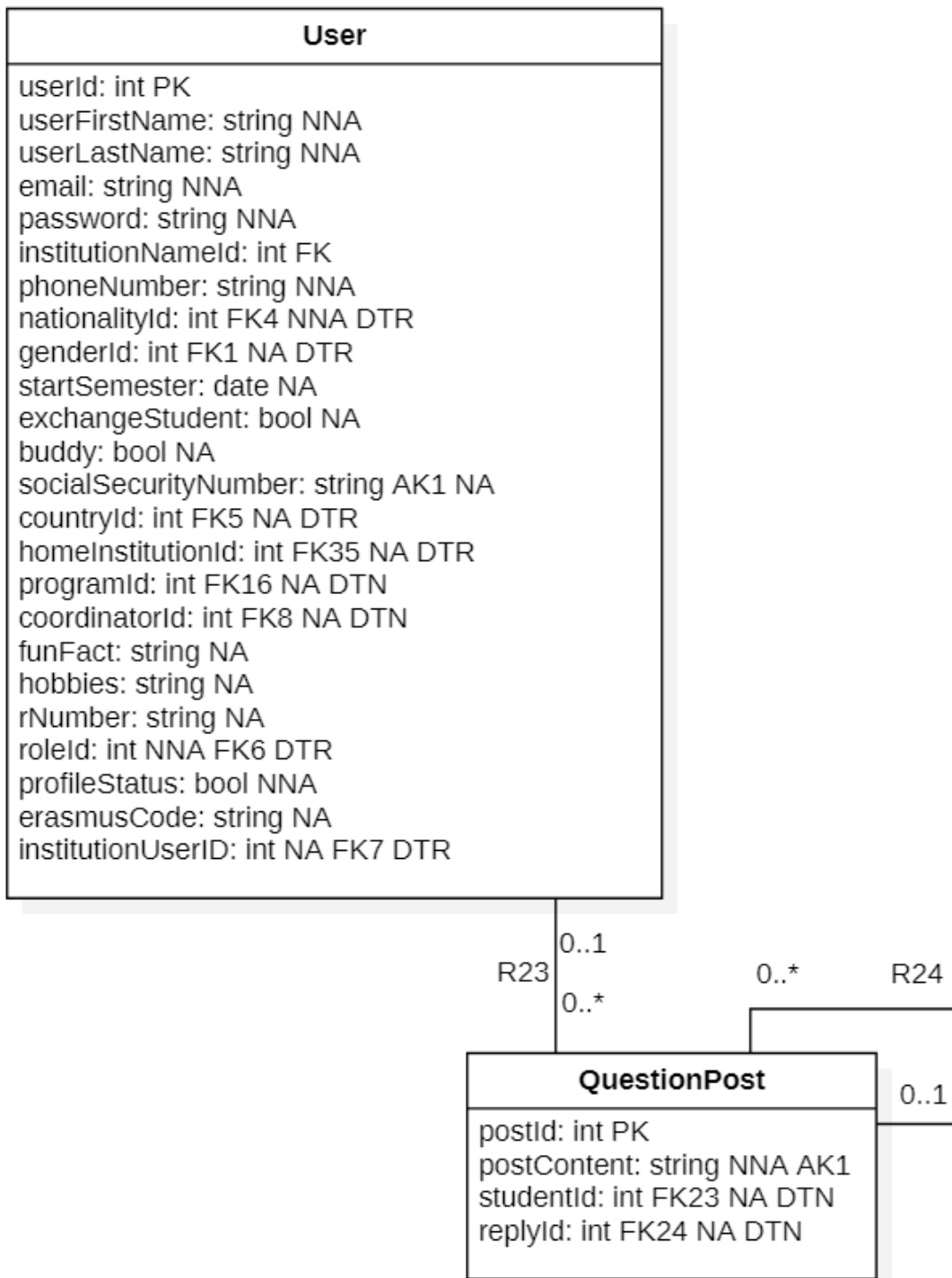
- **Property selected:** Based on the selected property we **READ** the “Room”, and “LandLordImage” table for information specific to every room like address, price, images, etc.
- **Room selected:** After a room has been selected we **READ** the “User”, “Document”, and “DocumentType” tables to display the landlord of the selected rooms information to the user.
- **Apply button clicked:** After the button has been clicked we **UPDATE** the “studentId” attribute for the record of the chosen rentable room in “Rent” based on the “userId” of the currently logged in user.

1.1.2.24. *Ask a question.*

Functionality: As a student, I can ask a question.

Precondition: Logged in as a student.

Data model view:



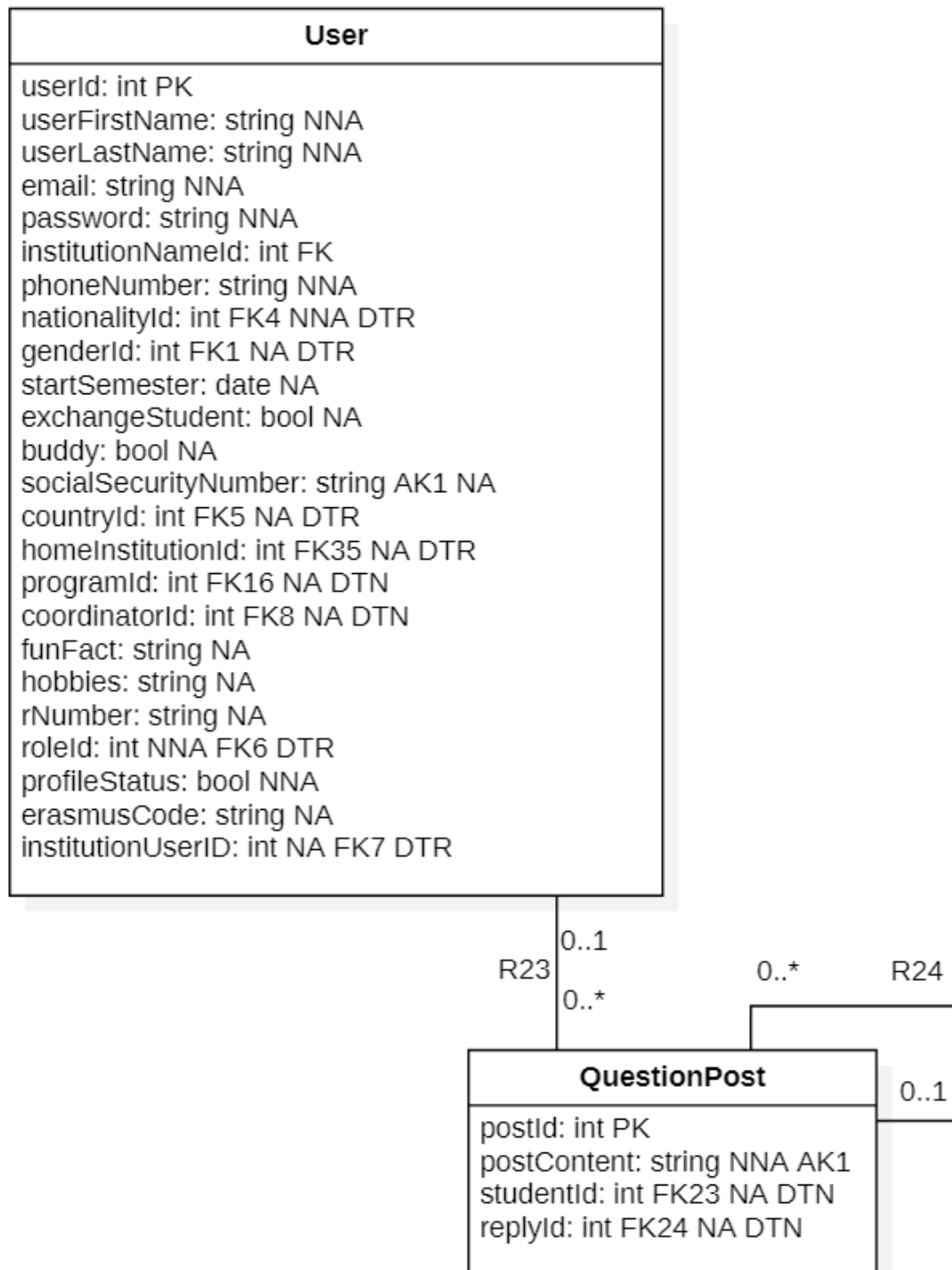
Link to prototype: [SMO-A New - Student](#)

1.1.2.25. *Reply to question.*

Functionality: As a student, I can reply to a question of another student.

Precondition: Logged in as a student.

Data model view:



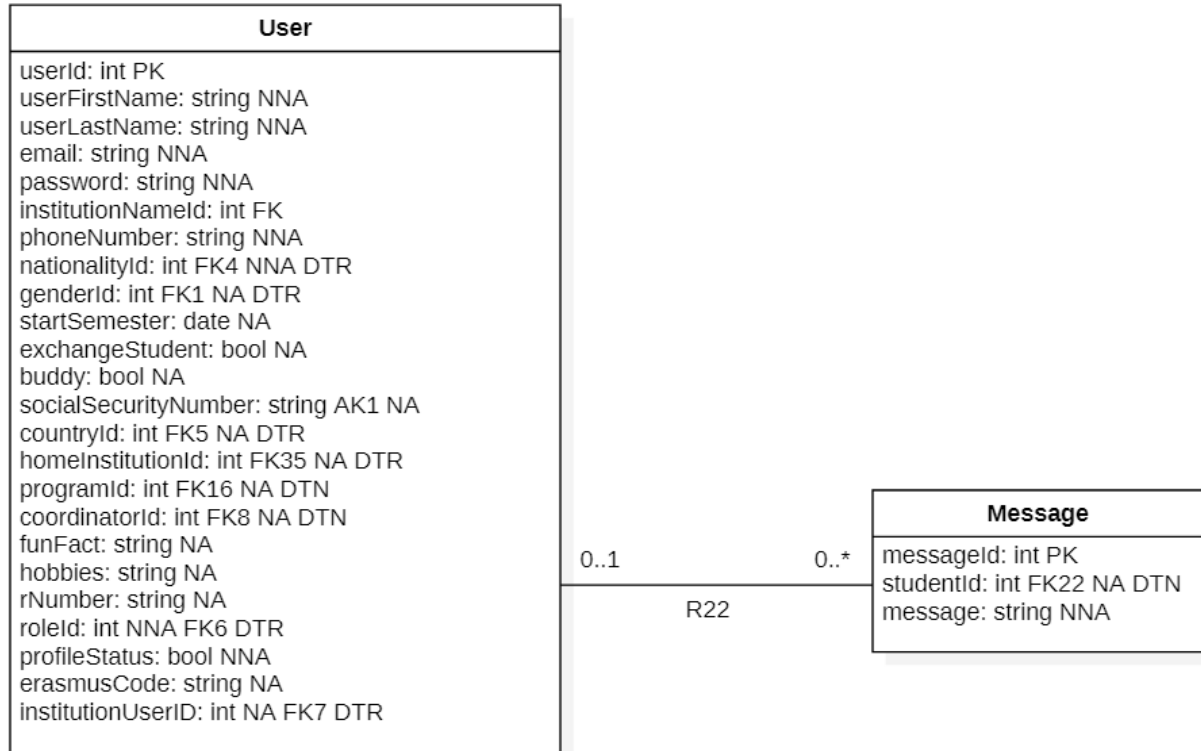
Link to prototype: [SMO-A New - Student](#)

1.1.2.26. *Send a message in board.*

Functionality: As a student, I can send a message in the community board.

Precondition: logged in as a student.

Data model view:



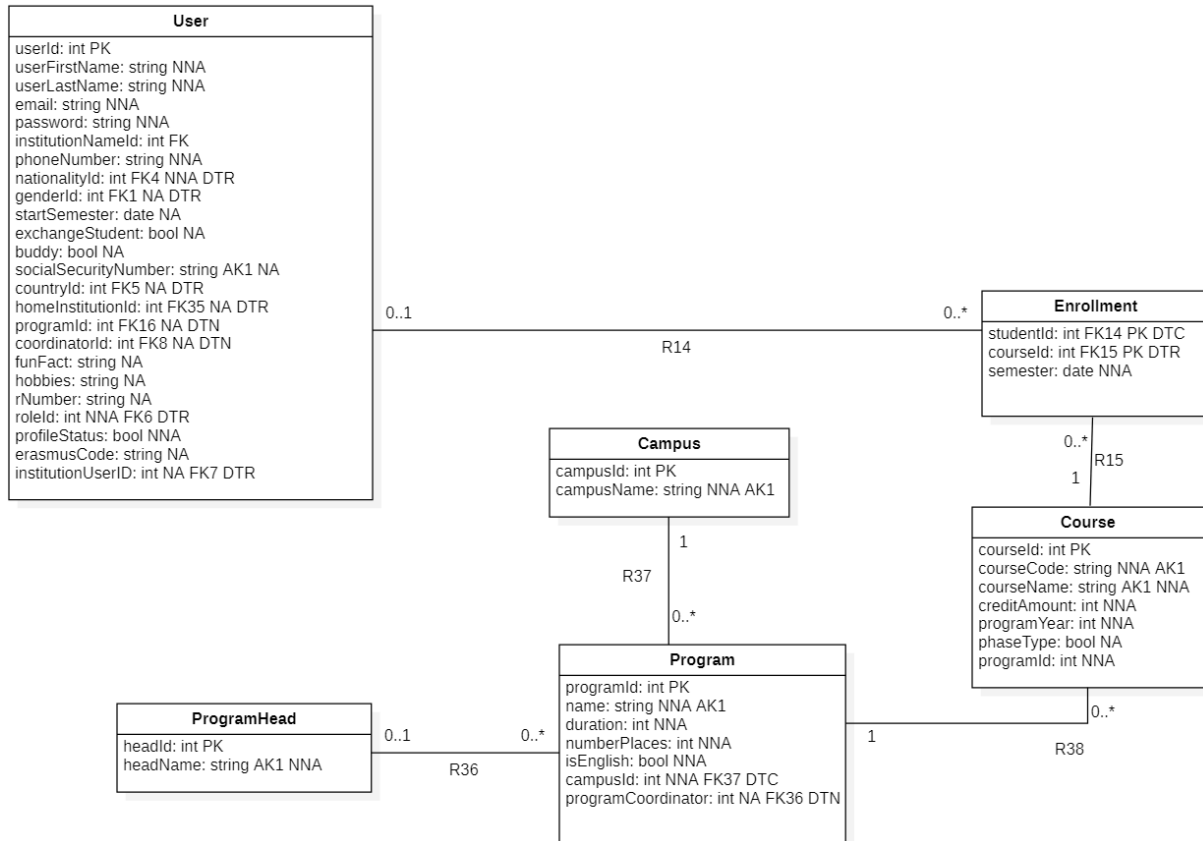
Link to prototype: [SMO-A New - Student](#)

1.1.2.27. *Enroll in courses*

Functionality: As an exchange student, I can enroll in courses.

Precondition: Logged in as student.

Data model view:



Link to prototype: [SMO-A New - Student](#)

Data model actions:

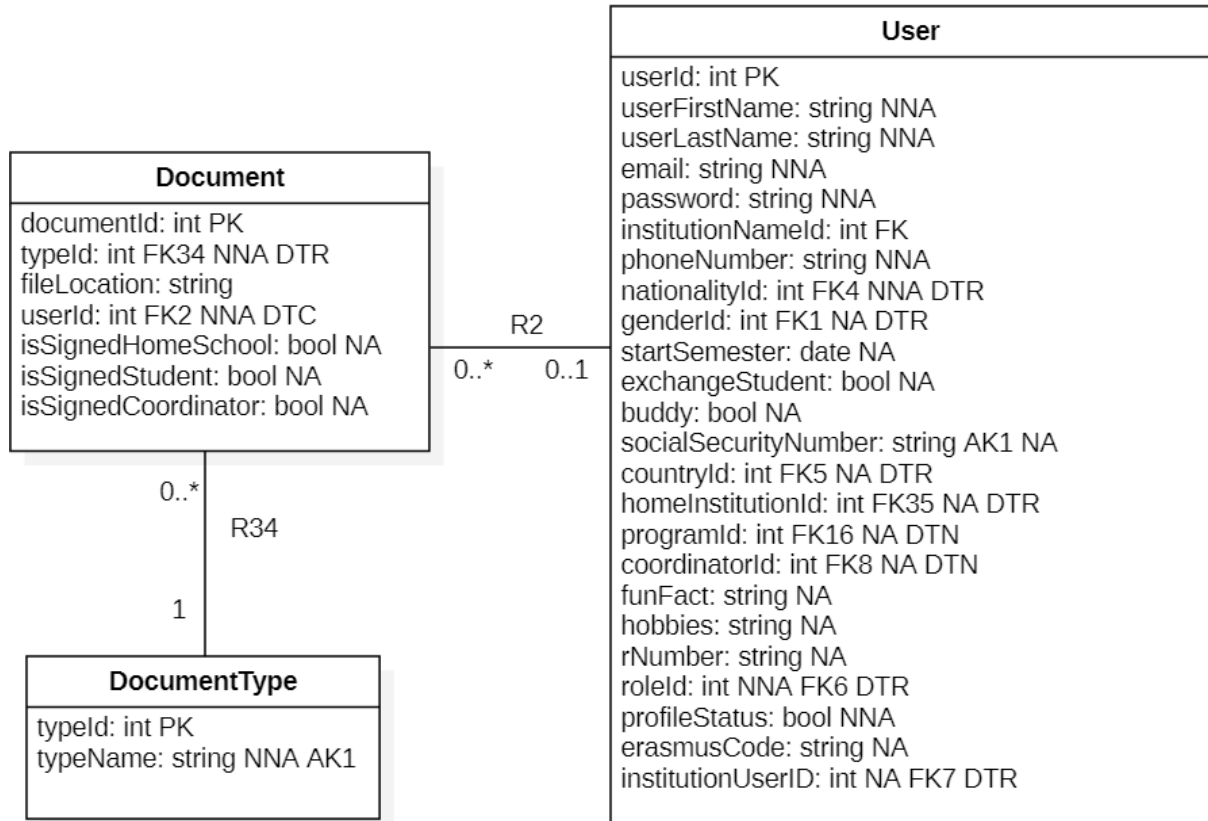
- **Initial state:** **READ** all available classes from the course table connected to the correct program
 - **Ticking courses:** **READ** number of credits for the class and change number of total credit (calculated attribute)
 - **Clicking save button:** **UPDATE** enrollment table of student with the chosen classes

1.1.2.28. Sign OLA

Functionality: As an exchange student, I can sign OLA.

Precondition: Logged in as student, OLA available to be signed.

Data model view:



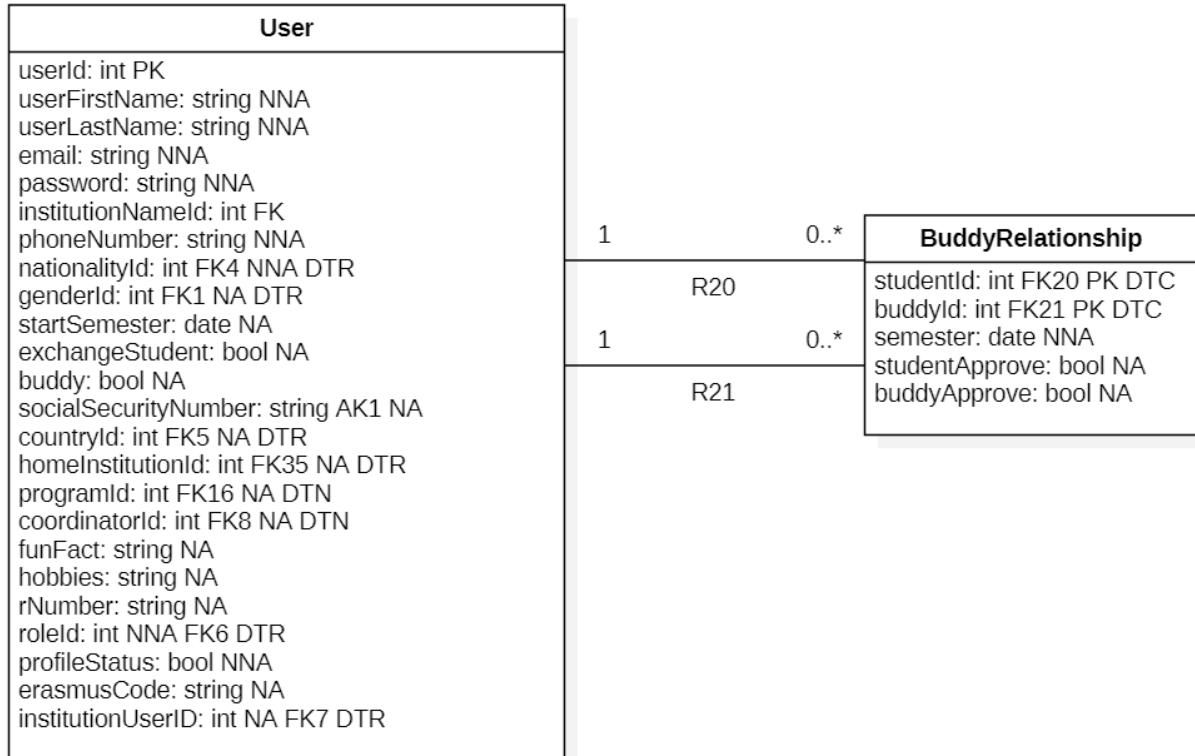
Link to prototype: [SMO-A New - Student](#)

1.1.2.29. View students

Functionality: As a buddy, I can view all the students who I am currently a buddy off..

Precondition: Logged into account with tag “buddy”.

Data model view:



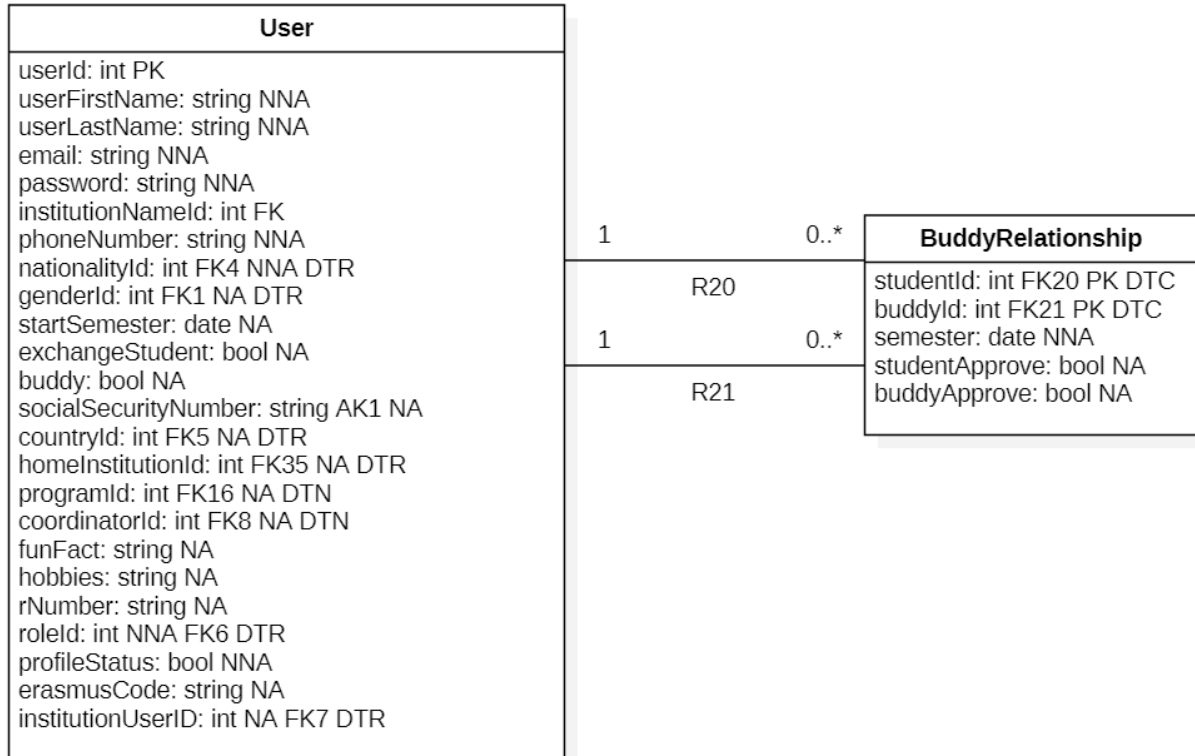
Link to prototype: [SMO-A New - Student](#)

1.1.2.30. Approve student

Functionality: As a buddy, I can approve or reject student

Precondition: Logged into account with tag “buddy”

Data model view:



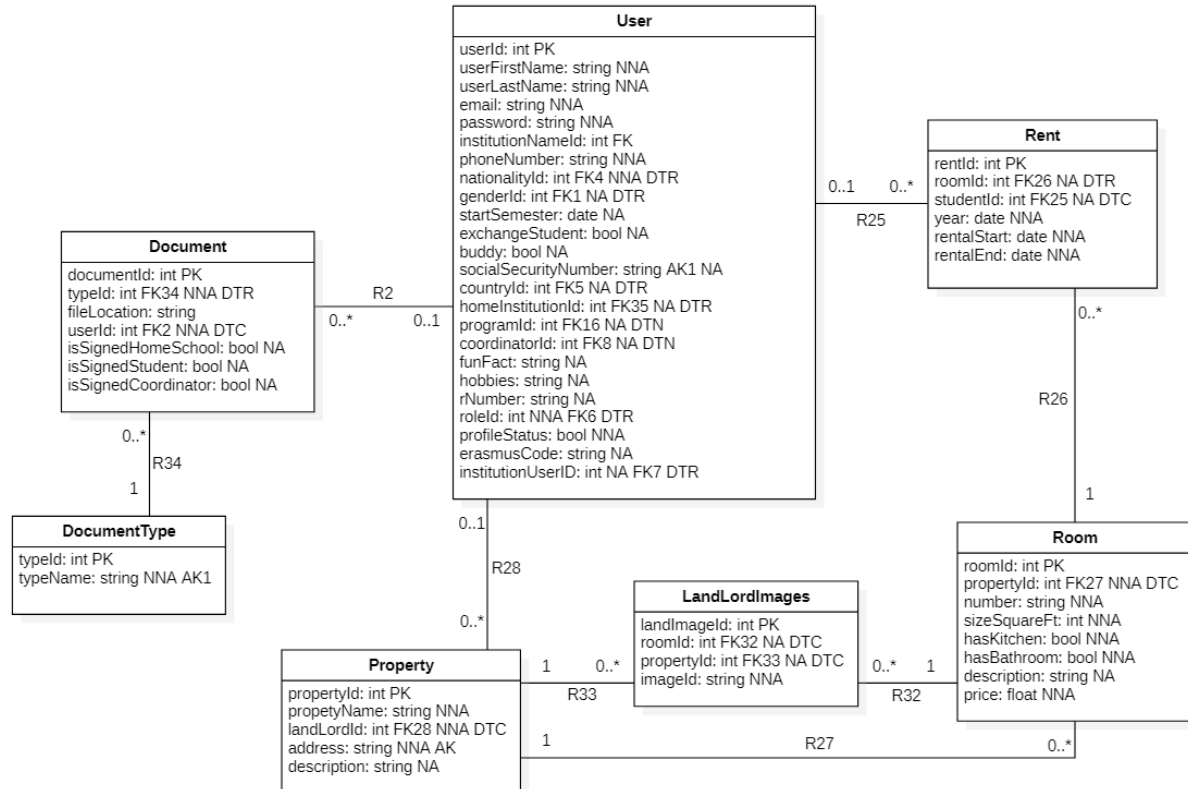
Link to prototype: [SMO-A New - Student](#)

1.1.2.31. Manage room.

Functionality: As a house owner, I can manage rooms.

Precondition: Logged in as landlord.

Data model view:



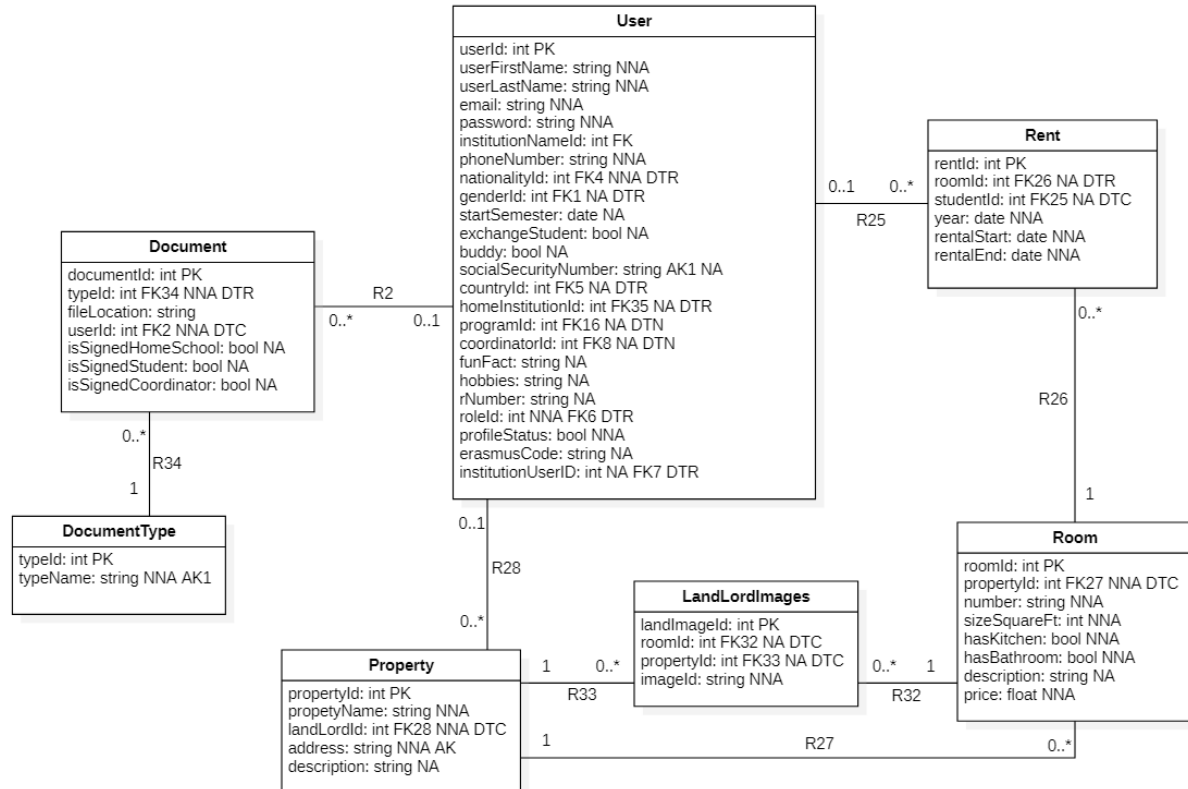
Link to prototype: [SMO-A New - Landlord](#)

1.1.2.32. *Manage properties.*

Functionality: As a house owner, I can manage properties.

Precondition: Logged in as landlord.

Data model view:



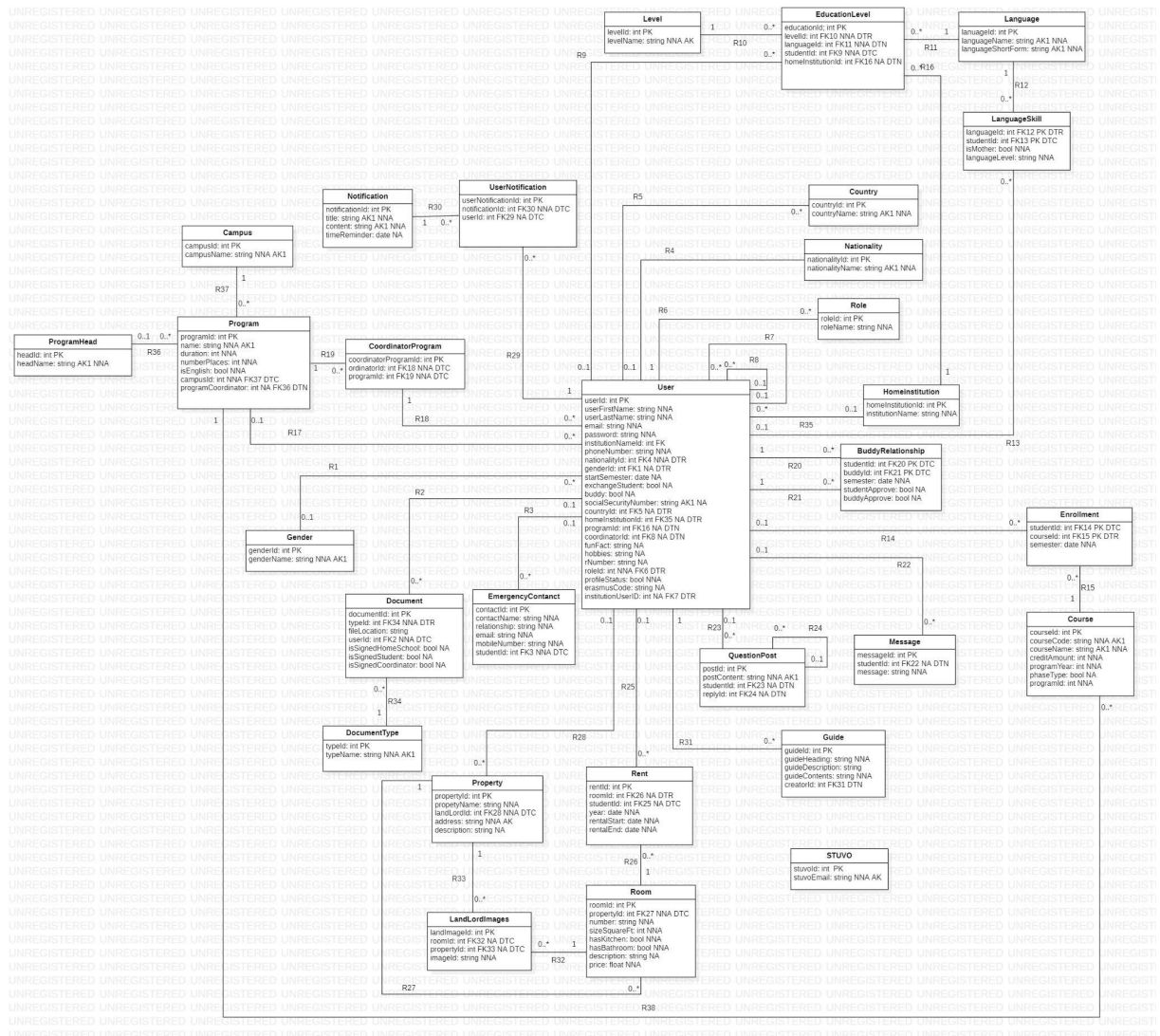
Link to prototype: [SMO-A New - Landlord](#)

1.1.2.33. Manage users.

Functionality: As an admin, I can manage users.

Precondition: Logged in as admin.

Data model view:



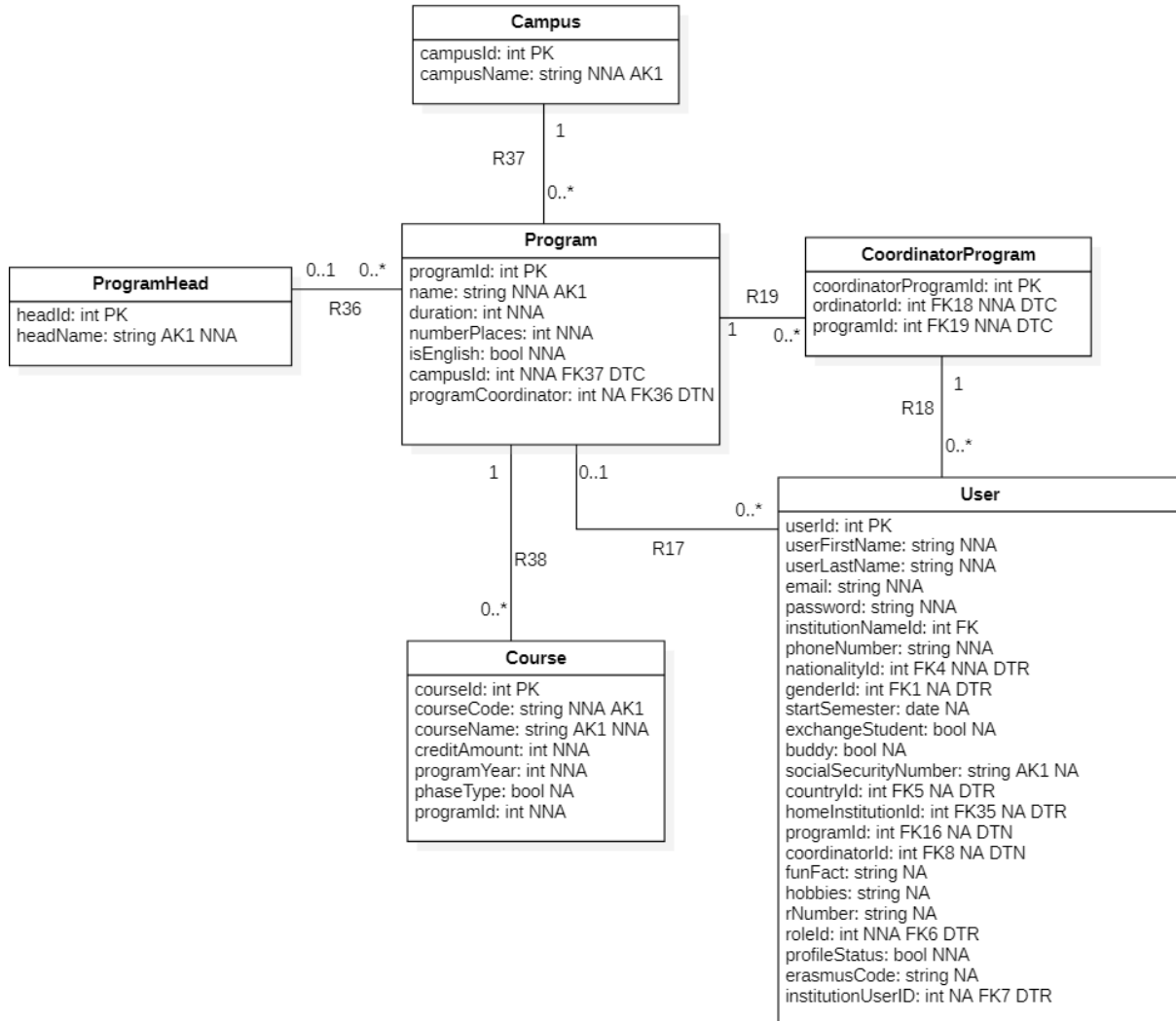
Link to prototype: [SMO-A New - Admin](#)

1.1.2.34. *Manage programs.*

Functionality: As an admin, I can manage programs.

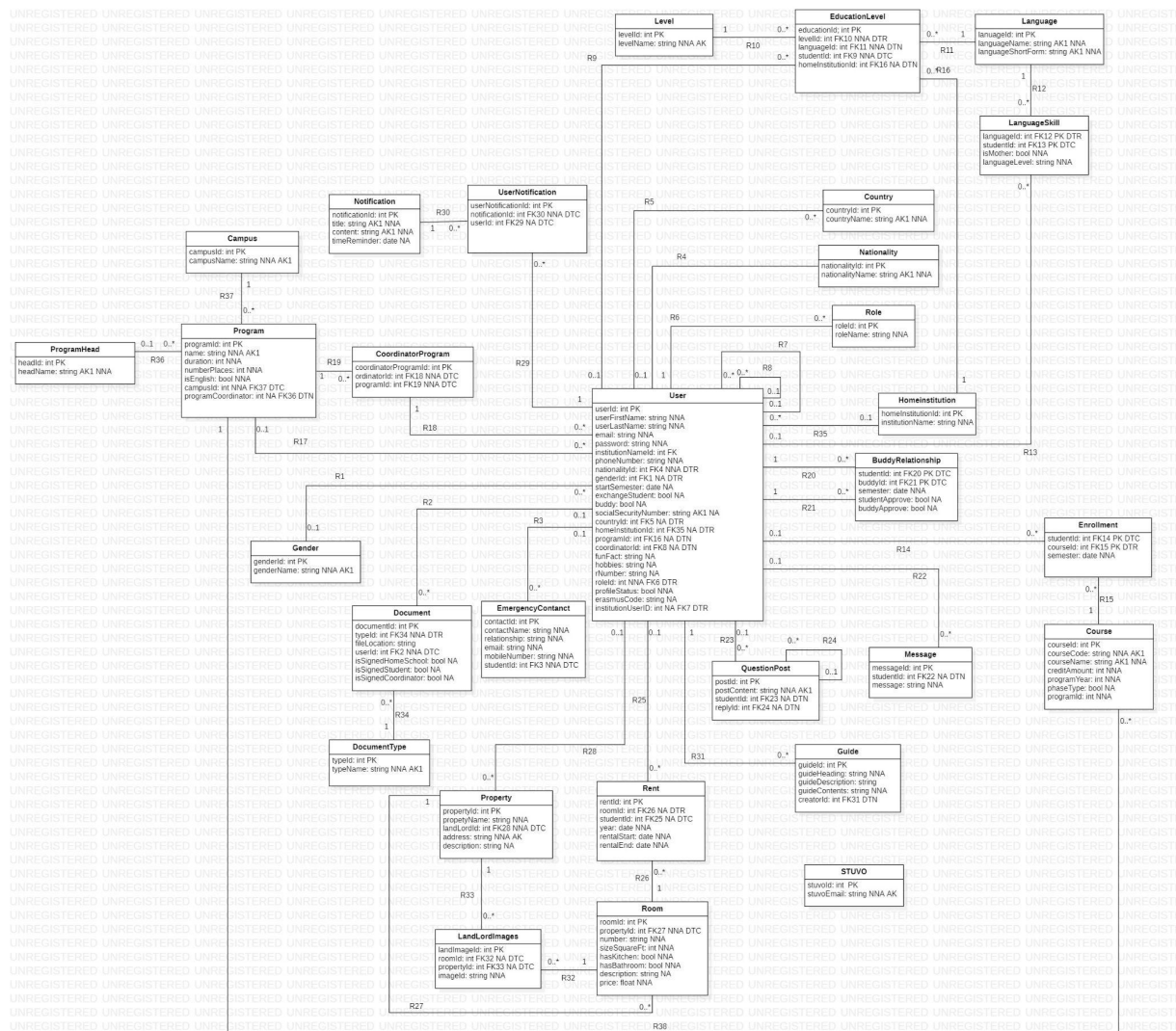
Precondition: Logged in as admin.

Data model view:



Link to prototype: [SMO-A New - Admin](#)

2. Data Model



Relationships:

R1: A user must have only one gender; there can be several users from the same gender. A gender cannot be deleted as long as there are students associated with it.

R2: A user can have several documents; a document belongs to only one user. If a user is deleted, their documents will also be deleted.

R3: A student can have several emergency contacts; an emergency contact belongs to only one student. If a student is deleted, their emergency contacts will also be deleted.

R4: A user must have a nationality; there can be several users with the same nationality. A nationality cannot be removed as long as there are students with that nationality.

R5: A student was born in a country; there can be several students from the same country. A country cannot be deleted as long as there are students who were born in that country.

R6: A user must have a role; many users can have the same role. A role cannot be deleted as long as there are users with that role.

R7: A student comes from a home institution; several students can come from the same home institution. A home institution cannot be deleted as long as there are students coming from that home institution.

R8: A student has one coordinator; one coordinator can have zero or many students. If a coordinator is deleted, the student keeps on existing without a corresponding coordinator.

R9+R10: A student can have done several educational levels; a level can be done by many students. If a student is deleted, the record of their educational levels will be deleted as well. A level cannot be deleted as long as there are records of a student who has that level.

R11+R16: A home institution can teach in several languages; one language can be used in many home institutions. If a language is deleted, the record of the home institution will keep on existing without a specified language. If a home institution is deleted, the record of their language will be deleted as well.

R12+R13: A student can know many languages; one language can be known by many students. A language cannot be deleted as long as there are records of students speaking it. If a student is deleted, the records of their spoken languages will be deleted as well.

R14+R15: A student can enroll in many courses; there can be many students enrolled in a course. If a student is deleted, the record of their class schedule gets deleted as well. A course cannot be deleted as long as there are records of students who take it.

R17: A student can be enrolled in only one program; many students can be enrolled in the same program. If a program is deleted, then student will not have a corresponding program anymore.

R18+R19: A coordinator can watch over many programs; a program can have several coordinators. If a program is deleted, the record of the corresponding coach will be deleted as well. If a coordinator is deleted the record of their programs will keep on existing without a coordinator.

R20+R21: A student can have zero or more buddies; one buddy can have several students. If a student is deleted, the record of their buddies will be deleted as well. If a buddy is deleted, the record of their students will be deleted as well.

R22: A student can send many messages; a message is sent by only one student. If a student is deleted, their messages will keep on existing without a corresponding student.

R23: A student can create many question posts; a question post belongs to only one student. If a student is deleted, their questions will keep on existing without a corresponding student.

R24: There can be several replies on a question post; a reply belongs to only one question. If a reply is deleted, their question will keep on existing without the reply.

R25+R26: A student can rent multiple rooms; a room might be rented by several students (in different years). If a student is deleted the record of their room will be deleted as well. A room cannot be deleted as long as there is a record of a student occupying it.

R27: A room is located on a property; a property can have several rooms. If a property is deleted, all the corresponding rooms will be deleted as well.

R28: A landlord can have several properties; a property must have a landlord. If a landlord is deleted, all their properties will be deleted as well.

R29+R30: A user can get many notifications; a notification can be sent to several users. If a user is deleted, the record of the user's notifications will be deleted as well. If a notification is deleted, the record of the user the notification was sent to, will be deleted as well.

R31: A user can create several guides; a guide is created by a user. If the creating user is deleted their guides will keep on existing without the creator.

R32: There can be several pictures of a room; a picture belongs to only one room. If a room is deleted all the pictures of it will be deleted as well.

R33: There can be several pictures of a property; a picture belongs to only one property. If a property is deleted all the pictures of it will be deleted as well.

R34: A document must have a type; there can be several documents with the same type. A type cannot be deleted as long as there are documents of that type.

R35: A student can have one home institution; there can be several students from the same institution. An institution cannot be deleted as long as there are students from it.

R36: A program is organized by a program head; a program head can organize several programs. If a program head is deleted, their programs will keep on existing without a specified head.

R37: A program is done only on one campus; there can be several programs on a campus. If a campus is deleted, all the programs done there will be deleted as well.

R38: There can be several courses in a program; a course is part of only one program. If a program is created, all their corresponding courses will be deleted as well.

3. User Stories

Send application form:

- As a user, I want to be able to fill in an application, so i can become a degree student
- As a user, I want to upload required documents (e.g., transcripts, ID, motivation letter) along with my application, so my submission is complete.
- As a user, I want to be notified about the status of my application, so I know whether it is accepted, or rejected.

Nominate student:

- As a home institution, I want to be able to nominate a student, so they can become an exchange student

Approve student(IO):

- As an International Office employee, I want to be able to see an overview of all the students, so I can quickly find the students I need to approve or reject.
- As an international Office employee, I want to be able to approve or reject students, so I can validate if their nomination is valid.
- As an International Office employee, I want to filter students by status (e.g., pending, approved, rejected), so I can quickly find the ones I need to process.
- As an International Office employee, I want to receive notifications when new students need approval, so I can process them on time.

Approve student profile:

- As a coordinator I need to be able to see an overview of all nominated students, so I can search for the student's profile which I need to accept or reject.
- As a coordinator, I need to be able to accept or reject a specific student's nomination, so I can have control and keep track of if the program has enough seats.
- As a coordinator, I want to receive notifications when new students need approval, so I can process them on time.
- As a coordinator, I want to see a detailed profile of each student, for example academic background, so I can make an informed decision.

Enroll in courses:

- As an exchange student, I want to be able to see an overview of all the courses I am able to take in my chosen program, so I can select them without issue or confusion.
- As an exchange student, I want to be able to select courses for my OLA, so my OLA can be created and signed.
- As an exchange student, I want to receive a confirmation once my course enrollment is finalized, so I know I am officially registered.

Ask a question:

- As a student, I want to be able to ask a question in the Q&A board, so if I need help I can ask someone.
- As a student, I want to be able to see the questions asked by others, so I can find answers easily.

Reply to a question(ST):

- As a student, I want to be able to see the Q&A board, so I can see if any questions need to be answered.
- As a student, I want to be able to reply to a question, so I can help the student with their question.

Sign OLA(HI):

- As a Home Institution, I want to be able to sign the OLA of the student I nominated, so I can make sure they get into all their courses.

Sign OLA(CO):

- As a Coordinator, I want to be able to sign the OLA of one of my students, so they can get into the courses they want.

Sign OLA(ST):

- As a student, I want to be able to sign the OLA, so I can get into all of the courses I selected.
- As a Student, I want to receive a notification once all signatures are completed, so I know my enrollment is final.

List of Epic's:

> ☐ Login	> ☐ Apply for room	> ☐ Manage home owner
> ☐ Apply as landlord	> ☐ Ask a question	> ☐ Plan webinar
> ☐ Apply as buddy	> ☐ Reply to a question(ST)	> ☐ Manage student
> ☐ Apply as home institution	> ☐ Send a message in board	> ☐ Approve student profile
> ☐ View homepage	> ☐ Enroll in courses	> ☐ Sign OLA(CO)
> ☐ Send application form	> ☐ Sign OLA(ST)	> ☐ Approve course enrollment
> ☐ Nominate Student	> ☐ View student	> ☐ Reply to a question(CO)
> ☐ Sign OLA(HI)	> ☐ Approve student(BD)	> ☐ Apply for buddy
> ☐ Approve home owner	> ☐ Manage rooms	> ☐ Approve buddy
> ☐ Approve student(IO)	> ☐ Approve Rental	> ☐ View guide
> ☐ Manage guide	> ☐ Manage properties	> ☐ Apply bike
	> ☐ Manage user	
	> ☐ Manage programs	